Table of Contents

Day 1: Definition 7

Day 2: AI vs ML vs Deep Learning 8

Day 3: Types of Machine Learning 8

1. Supervised Learning 8

2. Unsupervised Learning 9

3. Semi-Supervised Learning 10

4. Reinforcement Learning 11

Quick Comparison Table 11

Day 4: Batch Learning vs Online Learning (Machine Learning Training Styles) 12

A. Batch Learning 12

Day 5: Batch Learning vs Online Learning 12

B. Online Learning 12

C. Batch vs Online Learning Comparison 14

Day 6: Instance-Based Learning vs Model-Based Learning 14

A. Instance-Based Learning (Lazy Learning) 14

B. Model-Based Learning 15

C. Instance vs Model-Based Comparison 16

Summary Points 16

Day 7: Challenges in Machine Learning (ML) 17

Day 8: Applications of Machine Learning 17

Day 9: ML Development Life Cycle (ML DLC) 18

Day 10: Data-Related Job Roles 18

Database vs Data Warehouse (Simple) 19

Day 11: Tensors 19

Quick Practical Summary 20

Day 12: Virtual Environment 20

Day 15: Data Gathering 21

1. Working with CSV/TSV 21

Day 16: Working with JSON / SQL 22

Day 17: Fetching data from APIs 22

Day 18: Web scraping 22

Day 19: Understanding gathered data 22

Asking basic questions: 22

Day 20: Understanding gathered data 24

Exploratory Data Analysis (EDA) **24**
Univariate Analysis 24
A. Categorical Data 24
B. Numerical Data 24
Other Plots 25
Basic Stats 25
Day 21: Understanding gathered data ***25***
Exploratory Data Analysis (EDA) **25**
Bivariate Analysis 25
Day 22: Panda's profiling tool ***29***
Day 23: Feature Engineering (IMP) ***30***
1. FEATURE TRANSFORMATION **31**
2. FEATURE CONSTRUCTION **31**
3. FEATURE SELECTION **32**
4. FEATURE EXTRACTION **32**
Day 24: FEATURE SCALING (FEATURE TRANSFORMATION) ***32***
1. STANDARDIZATION **33**
Day 25: FEATURE SCALING ***34***
2. NORMALIZATION **34**
a. Min Max Scaling 34
b. Mean Normalization 34
c. Max Absolute Scaling 35
d. Robust Scaling 35
Extra 36
~~**Decimal Scaling Normalization**~~ 36
~~***StandardScaler and fit_transform Explained***~~ ***37***
~~**Purpose**~~ 37
~~**Line we are studying**~~ 37
~~**Step-by-step working**~~ 37
~~**Why this is important**~~ 38
Day 26: Encoding Categorical Variables ***38***
ORDINAL ENCODING **39**
LABEL ENCODING **39**
Day 27: Encoding Categorical Variables ***39***
One Hot Encoding **39**
Day 28: Column Transformer ***41***
Day 29: Pipeline Transformer ***42***
Day 30: Function Transformer ***44***
Common Transformations **44**
How to Check if Data is Normal **46**
Day 31: Power Transformer ***46***

Types of Power Transformer **47**
How Power Transformer Works **47**
When to Use Which **48**
DAY 32: BINNING AND BINARIZATION ***48***
TYPES OF BINNING **49**
1. UNSUPERVISED BINNING 49
2. SUPERVISED BINNING 50
3. CUSTOM BINNING 50
BINARIZATION **51**
DAY 33: HANDLING MIXED DATA ***51***
DAY 34: HANDLING DATE AND TIME VARIABLES ***52***
DAY 35: HANDLING MISSING VALUE ***53***
Removing values **53**
DAY 36: HANDLING NUMERICAL MISSING VALUE (Imputing) ***54***
4 techniques **54**
DAY 37: HANDLING CATEGORICAL MISSING VALUE ***55***
Two techniques: **55**
DAY 38: HANDLING MISSING VALUE ***56***
Random Sample Imputation: **56**
Missing indicator: **56**
Automatic selection of parameter: **57**
DAY 39: HANDLING MISSING VALUE (MULTIVARIATE) ***57***
Two techniques: **57**
KNN IMPUTER: **57**
DAY 40: HANDLING MISSING VALUE (MULTIVARIATE) ***58***
Iterative imputer - MICE: **58**
~~EXTRA: HOW TO~~ CHOOSE BEST METHOD 59
DAY 41: HANDLING OUTLIERS ***60***
Introduction **60**
A. Algorithms highly affected by outliers **60**
B. Algorithms less affected by outliers **61**
How to detect outliers **61**
A. Normal Distribution: 61
B. Skewed Distribution: 61
C. Any Distribution 61
How to treat outliers **61**
A. Trimming: 61
B. Capping or Winsorization: 62
C. Treat outliers as missing 62

D. Discretization 62
Techniques for outliers detection and removal: 62
DAY 42: HANDLING OUTLIERS 63
Outliers Removal And Detection Using Z Score: 63
DAY 43: HANDLING OUTLIERS 64
Outliers Removal And Detection Using IQR Method: 64
DAY 44: HANDLING OUTLIERS 65
Outliers Removal and Detection Using Winsorization: 65
DAY 45: FEATURE CONSTRUCTION | FEATURE SPLITTING 66
DAY 46: CURSE OF DIMENSIONALITY 67
DAY 47: PRINCIPLE COMPONENT ANALYSIS (PCA) 68
Geometric Intuition 68
EXTRA 68
DAY 48: PRINCIPLE COMPONENT ANALYSIS (PCA) 74
Problem Formulation and code solution 74
DAY 49: PRINCIPLE COMPONENT ANALYSIS (PCA) 77
PCA of MNIST Dataset along with code: 77
DAY 50: MACHINE LEARNING ALGORITHMS 79
1. Simple Linear Regression (Supervised- Regression) 79
What is “linear data” 79
Types of Linear Regression 80
Equation of a line 80
DAY 51: MACHINE LEARNING ALGORITHMS 93
1. Simple Linear Regression (Mathematical Intuition) 93
3. Closed Form Solution (OLS) 94
4. Gradient Descent (Non-Closed Form) 94
11. Your Code Explanation (Manual Linear Regression) 96
Code Intuition 97
DAY 52: MACHINE LEARNING ALGORITHMS 98
1. Regression Metrics (Linear Regression) 98
Error 98
Mean Absolute Error (MAE) 99
Mean Squared Error (MSE) 100
Root Mean Squared Error (RMSE) 101
R^2 Score (Coefficient of Determination- goodness of fit) 102
Adjusted R^2 Score 104
DAY 53: MACHINE LEARNING ALGORITHMS 105
Multiple Linear Regression (Geometric Intuition) 105
DAY 54: MACHINE LEARNING ALGORITHMS 106
Multiple Linear Regression (Mathematical formulation) 106

DAY 55: MACHINE LEARNING ALGORITHMS ***107***

Multiple Linear Regression (Code) **107**

DAY 56: MACHINE LEARNING ALGORITHMS ***108***

Assumptions of Linear Regression **108**

1. Linear Relationship between input and output 108
2. No multicollinearity 108
3. Normality of residual 108
4. Homoscedasticity 108
5. No autocorrelation of errors 108

DAY 57: MACHINE LEARNING ALGORITHMS ***109***

Gradient Descent from scratch **109**

OLS (Ordinary Least Squares) ***109***

Method 109
What is happening 109
Pros 109
Cons (important) 109
Best for 110

Gradient Descent ***110***

Method 110
What is happening 110
Pros 110
Cons 111
Best for 111

Core Difference (1 line) ***111***

🎯 Linear Regression: 117
🧭 Gradient Descent: 117

DAY 58: MACHINE LEARNING ALGORITHMS ***119***

Batch Gradient Descent **119**

Types on basiss of what: **120**

In batch we use total data total rows to update/calculate the slope **120**

It is very slow and have some computational problem **120**

For that we have stockistic **120**

We update based on rows , fast good for large datasets **120**

Mini batch we define a batch **120**

Make me understand the difference between the three types based on some example analogy **120**

Batch is rare and used when we have convex function but there dataset should be big **120**

Stochastic is used widely **120**

Gradient descent applying to multiple linear regreesion **120**

Mathematical formulation: **120**

Y= b0+b1x+b2x **120**

Steps **120**

We start with random values of b0 =0 and slope b1 , b2 = 1 ..... **120**

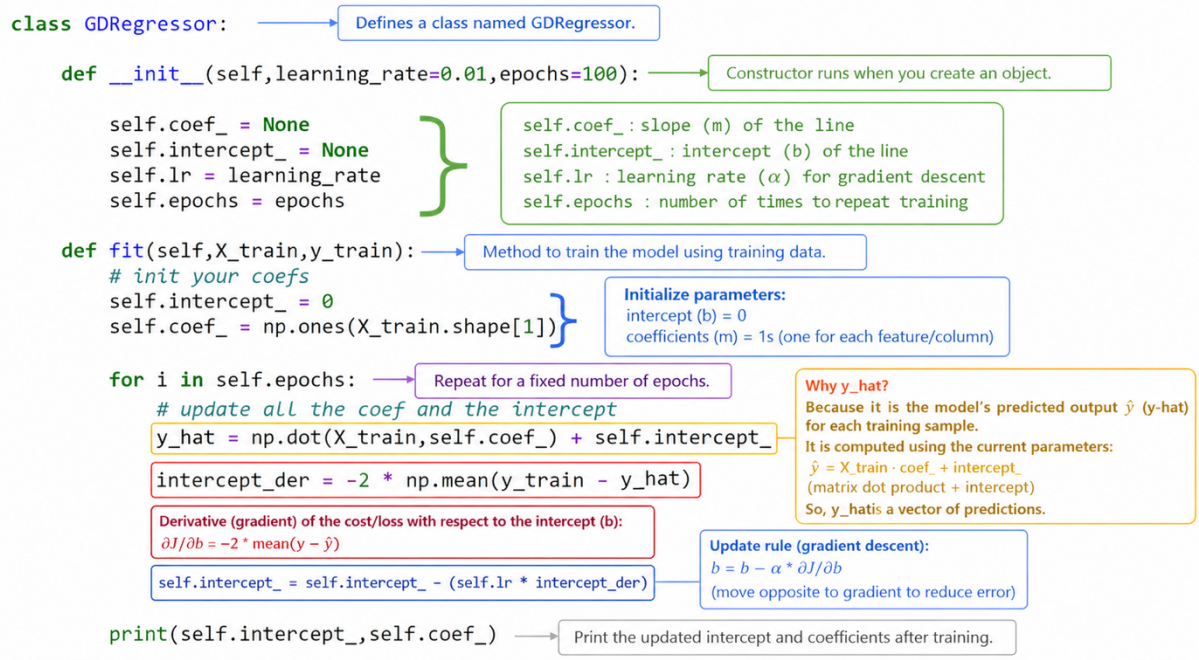
B0=b0 – n slope **120**

B1=b1 – n slope **120**

B2=b2 – n slope **120**

How to calculate the slope of intercept **120**

We have loss function MSE which we gonna expand **120**



..... **122**

DAY 59: MACHINE LEARNING ALGORITHMS ***123***

Stochastic Gradient Descent **123**

DAY 60: MACHINE LEARNING ALGORITHMS ***125***

Mini Batch Gradient Descent **125**

Group of rows here , in between batch and stochastic gd **125**

We create batches here like 100 rows and one batch is of 10 rows so total 10 batches we got **125**

Now in each epoch we do updates equal to the number of the batch **125**

DAY 61: MACHINE LEARNING ALGORITHMS ***126***

Polynomial Linear Regression **126**

DAY 62: MACHINE LEARNING ALGORITHMS ***129***

Bis Variance Trade off **129**

Techniques to handle this 130

DAY 70: MACHINE LEARNING ALGORITHMS ***132***

Logistics Regression (Perceptron Trick) **132**

Two approaches 134

What is perceptron trick 134

DAY 71: MACHINE LEARNING ALGORITHMS ***136***

Logistics Regression (Perceptron Trick Code).....136
DAY 72: MACHINE LEARNING ALGORITHMS.....*136*
Logistics Regression (Sigmoid Function).....136

100 days of Machine Learning

Day 1: Definition

Machine Learning (ML)

Machine Learning is about teaching computers to learn from data instead of being explicitly programmed.
It uses statistical and mathematical techniques to find patterns and make predictions or decisions.

The key idea is **learning from experience (data)** and **improving automatically** as more data becomes available.
Unlike traditional programs, ML systems update their logic when new data changes, making them adaptive rather than static.

Algorithm:

- A procedure or method.
- A learning rule.
- Defines how to learn from data.
- Examples: Linear Regression, Decision Tree, KNN, SVM, Gradient Descent.

Model:

- The trained result of an algorithm.
- A mathematical function with learned parameters.
- Used to make predictions.
- Example: A specific line from Linear Regression with learned weights.

Example. Spam email filter.

Algorithm: Naive Bayes or Logistic Regression.

Model: A trained classifier.It outputs spam or not spam for new emails.

Rule.

- Algorithm learns.
- Model predicts.

Day 2: AI vs ML vs Deep Learning

- **Artificial Intelligence (AI)**
  - The broadest concept.
  - Any technique that enables machines to mimic human intelligence, reasoning, or decision-making.
  - Current AI is mostly pattern recognition, not true human-like intelligence.
- **Machine Learning (ML)**
  - A subset of AI.
  - Focuses on algorithms that learn from data.
  - Works well when features (input variables) are clearly defined.
  - Example: predicting house prices using features like size, location, and number of rooms.
- **Deep Learning (DL)**
  - A subset of ML.
  - Based on **neural networks** with multiple layers.
  - Automatically learns useful features from raw data (e.g., images, text, sound) without manual feature selection.
  - Excels in complex, unstructured data and fuzzy logic scenarios.
  - The deeper the network (more layers), the more powerful its ability to capture abstract patterns.
  - While traditional ML models reach a performance limit, DL models often improve with more data and compute power.

Summary Points

- AI is the broad goal of creating intelligent systems.
- ML is one approach to achieve AI through data-driven learning.
- DL is an advanced form of ML that uses neural networks to learn from raw data.
- ML requires feature engineering; DL learns features automatically.
- DL performs best when you have large amounts of data and computational power.

==========

Day 3: Types of Machine Learning

Machine Learning (ML) is divided based on **the level of supervision or guidance** provided to the algorithm during training.
"Supervision" means **whether we provide correct answers (labels)** to the model while it learns.

1. Supervised Learning

Definition:
In supervised learning, the algorithm learns from **labeled data** — both input features and their correct output (target) are known.

The goal is to learn a mapping from inputs (X) to outputs (Y), so that when new data comes, it can predict the correct output.

Labels are the **answers or outcomes** you want your machine learning model to predict. **Labeled data** = input + correct output. Supervised learning uses labeled data — both input and correct output — to learn mappings.

Example:
You give a dataset of houses (size, rooms, location) with their prices. The model learns the relation between features and price.

Common Types:

- **Regression:** Output is **numerical**.
  Example: Predicting salary(label), temperature, or price.
  "How many dogs are in the image?" → Regression (Linear Regression)
- **Classification:** Output is **categorical**.
  Example: Predicting "spam or not spam," "disease or healthy."
  "Is there a dog in the image?" → Classification (Logistic Regression)

Real-World Use Cases:

- Credit score prediction
- Email spam detection
- Disease diagnosis
- Stock price prediction

k-NN, Naïve Bayes, SVM, Decision Trees, Logistic Regression, and Random Forest are **supervised classification algo**.

Linear regression is regression algorithm

k-NN
• Classification → majority vote
• Regression → average of neighbors

Decision Tree
• Classification → class at leaf
• Regression → numeric value at leaf

SVM
• Classification → separating hyperplane
• Regression → best-fit function with margin

---

2. Unsupervised Learning

Definition:
In unsupervised learning, the data has **only inputs**, no labels.
The algorithm tries to find **patterns, structures, or relationships** within the data itself.

Common Types:

- **Clustering:**
  Groups similar data points into clusters based on similarity.
  Example: Customer segmentation (grouping users by buying behavior).
  Use Case: Marketing companies grouping customers into buying segments. Algorithm: K-Means, K-Medoids, Hierarchical clustering
- **Dimensionality Reduction:**
  Reduces the number of features (columns) while keeping the important information.
  Helps simplify data and speed up computation.
  Example: Reducing 100 features of face data into 10 key features using **PCA (Principal Component Analysis)**.
  Use Case: Image compression or visualization of high-dimensional data.
- **Anomaly Detection:**
  Detects unusual or rare data points (outliers).
  Example: Detecting credit card fraud or sensor failure.
  Use Case: Banking systems spotting suspicious transactions.
- **Association Rule Learning:**
  Finds relationships or associations between variables in large datasets.
  Example: "People who buy bread often buy butter." (Market Basket Analysis)
  Use Case: E-commerce recommendation systems. Apriori algorithm, FP Growth Tree, ECLAT

Real-World Use Cases:

- Customer segmentation (clustering)
- Fraud detection (anomaly)
- Recommendation systems (association)
- Feature reduction for visualization (PCA)

---

3. Semi-Supervised Learning

Definition:
Combination of supervised and unsupervised learning.
Some data is labeled, and the rest is unlabeled. The algorithm uses the small labeled portion to help label or understand the unlabeled data.

Why it's used:
Labeling large datasets is costly and time-consuming. Semi-supervised learning helps reduce that effort.

Example:
You label 100 animal images manually. The model then learns to label 10,000 more on its own.

Use Case:

- Self-driving cars: Only some road images are labeled, others are learned through similarity.

- Medical imaging: Doctors label a few X-rays, and the model learns from the rest.
- Voice recognition: Few recordings are labeled manually, the rest are learned automatically.

---

4. Reinforcement Learning

Definition:
In reinforcement learning (RL), an **agent** interacts with an **environment**, takes actions, and learns by receiving **rewards or penalties** based on its performance.
The goal is to **maximize total reward** over time.

Process:

1. The agent observes the environment.
2. It selects an action based on its current policy (rulebook).
3. It performs the action and receives feedback (reward or penalty).
4. It updates its policy to improve future actions.

Example:
A robot learns to walk by trial and error.
If it falls, it gets a penalty; if it walks correctly, it gets a reward.

Use Cases:

- Game playing (Chess, Go, Atari, etc.)
- Self-driving cars (learning to steer, accelerate, brake)
- Robotics (path planning, control systems)
- Recommendation systems (improving engagement over time)

Quick Comparison Table

Type	Data Used	Output	Key Algorithms / Concepts	Examples
Supervised	Labeled	Predict known target	Regression, Classification (Linear Regression, Decision Trees)	Spam detection, Price prediction
Unsupervised	Unlabeled	Find hidden patterns	Clustering (K-Means), PCA, Association	Market segmentation, Fraud detection
Semi-Supervised	Partially labeled	Predict + label new data	Self-training models	Self-driving, Medical imaging
Reinforcement	Interaction data	Sequence of actions	Q-Learning, Deep Q-Network	Games, Robotics, Auto-driving

==========

Day 4: Batch Learning vs Online Learning (Machine Learning Training Styles)

A. Batch Learning

Definition:
Batch (or offline) learning is a conventional way of training a machine learning model where **the entire dataset is used at once** to train the model.

How it works:

- You collect a full dataset.
- Train the model once using all data (offline).
- Deploy the trained model to production (server).
- The model stays static unless retrained manually with new data.

Example:

- Training a movie recommendation model using last year's user data, then deploying it.
- The model won't adapt automatically if user preferences change.

Flow:
Development → Train on full data → Save trained model → Deploy to production → Use for predictions.

Advantages:

- Simple and stable.
- Works well when data is static or changes slowly.
- Easier to implement and monitor.

Disadvantages:

- Model becomes outdated if new data trends appear.
- Retraining large models is time-consuming and costly.
- Not suitable for real-time adaptive systems.

Applications:

- Fraud detection systems updated monthly.
- Image recognition systems with stable data (like identifying handwritten digits).

Day 5: Batch Learning vs Online Learning

B. Online Learning

Definition:
Online (or incremental) learning is a training approach where the model learns **continuously or in small batches**, updating itself whenever new data arrives.

How it works:

- Data arrives one by one or in mini-batches.
- Model updates weights incrementally after each new observation.
- Learning and prediction happen together in real time.

Example:

- YouTube's recommendation system continuously learns from user watch history.
- Stock price prediction models update as new prices come in.
- Chatbots improve as they interact with more users.

Key Term – Concept Drift:
When the relationship between input and output changes over time (e.g., user behavior, market trends), the model needs to adapt continuously. Online learning helps handle this drift.

Advantages:

- Adapts to new data automatically.
- Ideal for dynamic environments.
- Reduces need for periodic retraining.

Disadvantages:

- Harder to implement and monitor.
- Risk of learning wrong patterns if data quality drops.
- Requires careful control of learning rate to balance new vs old data.

Learning Rate:
Defines **how fast the model updates** when new data arrives.

- High learning rate = model forgets old data quickly.
- Low learning rate = model adapts slowly.
  Good online models maintain a balance between memory and adaptation.

Out-of-Core Learning:
Technique for training on datasets too large to fit in memory.

- Data is divided into smaller chunks.
- Model learns incrementally using these chunks.
  Example: training on 100GB dataset by loading 1GB at a time.

Tools:

- **Batch Learning:** scikit-learn, TensorFlow, PyTorch.
- **Online Learning:** River, Vowpal Wabbit, `SGDRegressor` or `SGDClassifier` in scikit-learn.

Applications:

- Online ads click prediction.
- Real-time stock forecasting.
- Streaming data analysis.
- Dynamic content recommendations.

C. Batch vs Online Learning Comparison

Feature	Batch Learning	Online Learning
Data usage	Uses full dataset at once	Uses new data incrementally
Adaptability	Static	Dynamic
Training time	High (offline)	Continuous (low per update)
Complexity	Simple	Complex
Real-time updates	No	Yes
Use cases	Stable data, periodic updates	Streaming, fast-changing data
Tools	scikit-learn, TensorFlow	River, Vowpal Wabbit, scikit SGD models

Which one to pick?

- If your data changes slowly → **Batch Learning.**
- If your data changes frequently or streams in → **Online Learning.**

==========

Day 6: Instance-Based Learning vs Model-Based Learning

These are two **types of learning approaches** based on how the model learns from data.

A. Instance-Based Learning (Lazy Learning)

Definition:
Models that **memorize the training data** and make predictions by comparing new data with stored examples.

How it works:

- No generalization or pattern finding during training.
- During prediction, it measures similarity between input and stored data.
- Prediction is made based on "closest" stored instances.

Example:
K-Nearest Neighbors (KNN):

- Stores all data points.
- When predicting, finds the ‘k’ most similar points.
- Predicts based on majority vote (classification) or average (regression).

Other Examples:

- Radial Basis Function (RBF) networks.
- Kernel regression.

Advantages:

- Simple to implement.
- Works well with small datasets.
- No training time (only prediction time).

Disadvantages:

- Prediction is slow for large datasets (needs to compare with all stored points).
- High memory usage (stores entire data).
- Sensitive to noise and irrelevant features.

Applications:

- Pattern matching.
- Recommendation systems (similar user search).
- Anomaly detection with small data.

B. Model-Based Learning

Definition:
Models that **learn general patterns** from data and build a mathematical relationship between inputs and outputs.

How it works:

- Finds relationships (boundaries, weights, functions) during training.
- After learning, model parameters are stored (not the data).
- Predictions are fast and don’t need the full dataset again.

Example:
Linear Regression:
Learns the line (y = mx + b) that best fits the data.
Once trained, only coefficients (m, b) are saved, not the entire dataset.

Other Examples:

- Logistic Regression

- Decision Trees
- Support Vector Machines
- Neural Networks

Advantages:

- Efficient and fast for prediction.
- Requires less memory.
- Works well for large datasets.

Disadvantages:

- Requires time to train.
- Model might underfit or overfit.
- Less flexible if data pattern changes (unless retrained).

Applications:

- Credit scoring.
- Disease prediction.
- Image classification.
- Speech recognition.

C. Instance vs Model-Based Comparison

Feature	Instance-Based	Model-Based
Learning style	Memorizes data	Learns general patterns
Training time	Almost none	Requires training
Prediction time	Slow	Fast
Memory use	High	Low
Adaptability	Easy to add new data	Needs retraining
Examples	KNN, RBF	Linear Regression, SVM, Neural Nets
Real-world use	Small/simple datasets	Most real-world applications

Which one has more use?

- **Model-based learning** dominates in real-world systems.
- **Instance-based learning** is limited to smaller tasks or as part of hybrid systems.

Summary Points

- **Batch Learning:** Train once, static, easy, used when data is stable.
- **Online Learning:** Train continuously, adaptive, complex, used in dynamic systems.

- **Instance-Based Learning:** Memorize data, lazy, used for similarity-based predictions.
- **Model-Based Learning:** Learn patterns, generalize, used for most ML tasks today.

==========

Day 7: Challenges in Machine Learning (ML)

1. **Data Collection**
   - Sources: APIs, web scraping, databases.
   - Challenge: Collecting relevant and sufficient data.
2. **Insufficient / Labelled Data**
   - ML models need enough examples to learn patterns.
   - Labelled data is required for supervised learning.
3. **Unreasonable Effectiveness of Data**
   - With huge datasets, even simple algorithms perform well.
   - Quality matters less than quantity in some cases.
4. **Non-Representative Data**
   - Data may not reflect the real-world scenario.
   - Leads to **sampling noise** (random errors) and **sampling bias** (systematic errors).
5. **Poor Quality Data**
   - Missing values, duplicates, inconsistent data.
   - Cleaning can take more time than model training.
6. **Irrelevant Features**
   - "Garbage in, garbage out."
   - **Feature engineering**: create meaningful features, remove irrelevant ones.
7. **Overfitting**
   - Model memorizes training data but fails on new data.
   - Solution: regularization, more data, cross-validation.
8. **Underfitting**
   - Model too simple, fails to capture patterns.
   - Solution: more complex model, better features.
9. **Software Integration**
   - ML models must integrate with existing systems, APIs, or apps.
10. **Offline Learning / Deployment**
    - Some ML models cannot update in real-time; need batch updates.
11. **Cost Involved**
    - Computational resources, storage, maintenance.
    - **MLOps**: Automates deployment, monitoring, scaling, retraining.

Day 8: Applications of Machine Learning

1. **Retail**

    - Predict popular products, market basket analysis, recommendation systems (Amazon).
2. **Banking and Finance**
    - Fraud detection, credit scoring, stock prediction.
3. **Transport**
    - Route optimization, autonomous vehicles, demand prediction.
4. **Manufacturing**
    - Predictive maintenance (e.g., robotic arm in car production).
5. **Social Media**
    - Sentiment analysis (reviews, tweets).
    - Influences stock decisions, product recommendations.

==========

Day 9: ML Development Life Cycle (ML DLC)

1. **Problem Framing**
    - Define clear ML objectives and scope.
2. **Data Collection**
    - APIs, web scraping, databases, data warehouses.
3. **Data Pre-processing**
    - Cleaning, integration, transformation, reduction.
4. **Exploratory Data Analysis (EDA)**
    - Study input-output relationships.
    - Uni-variate, bi-variate, multi-variate analysis.
    - Detect outliers, handle imbalanced datasets.
5. **Feature Engineering & Selection**
    - Features = input columns.
    - Create new features by combining columns.
    - Select most important features.
6. **Model Training, Evaluation & Selection**
    - Try multiple algorithms.
    - Evaluate using metrics (accuracy, precision, recall).
    - Hyperparameter tuning to improve performance.
    - Ensemble learning: combine multiple models for better results.
7. **Model Deployment**
    - Deploy on servers (Heroku, AWS, etc.).
8. **Testing**
    - A/B testing to compare model versions.
9. **Optimization & Maintenance**
    - Model & data backups, automation, load balancing.
    - Retraining to prevent model decay.

==========

Day 10: Data-Related Job Roles

1. **Data Engineer**
    - Collect and prepare data, build pipelines.

- Skills: databases, ETL, backend systems.

2. **Data Analyst**
   - Summarize and visualize data, support decisions.
   - Skills: SQL, Excel, visualization tools.
3. **Data Scientist**
   - Combines engineering + statistics + ML modeling.
   - Full-stack data professional.
4. **ML Engineer**
   - Deploy ML models to production.
   - Focus on scaling, optimization, reliability.
5. **MLOps Engineer**
   - Automate ML lifecycle: deployment, monitoring, retraining.

ML Engineer / MLOps Engineer – Highest earning potential, strong demand, future-proof.

Data Scientist – Very high salary, versatile, stable demand.

Database vs Data Warehouse (Simple)

Feature	Database	Data Warehouse
Purpose	Store daily operational data	Store historical, analytical data
Data Type	Current/transactional	Aggregated, historical
Queries	Frequent, simple	Complex analytics
Users	Application developers	Data analysts, BI teams
Example	MySQL, PostgreSQL	Amazon Redshift, Snowflake

==========

Day 11: Tensors

container/ data structure for numbers.
Matrix is a tensor.
NumPy nd-array is a tensor.
ML uses mostly **0D to 5D tensors**.

Tensor Type	Meaning	Example	Axes / Dimensions	Shape	Where Used	Extra Explanation
0D Tensor (Scalar)	Single number. No axis.	`3, 7, 2`	0 axes	`()`	Basic arithmetic	A scalar is the smallest unit of data. No length, no row, no column.
1D Tensor (Vector)	List of numbers. One axis.	`[1,2,3,4,5]`	1 axis (length)	`(5)`	Features, tokens, embeddings	A vector has magnitude or direction in math. In ML it stores simple sequences.

2D Tensor (Matrix)	Table of numbers. Rows and columns.	`[[1,2,3], [5,6,7], [8,9,0]]`	2 axes (rows, columns)	`(3,3)`	Tabular data, weights, grayscale images	Most common tensor in data science. Each row is one sample or feature.
3D Tensor	Stack of multiple 2D matrices.	Sentence example: `[[1,0,0,0], [0,1,0,0]]`	3 axes	`(num_matrices, rows, columns)`	NLP, time-series, embeddings	Think of it as "multiple tables stacked on top of each other". Example: 3 sentences → `(3,2,4)`.
4D Tensor	Stack of 3D tensors.	Images: `(N,H,W,C)`	4 axes	`(batch, height, width, channels)`	CNNs, image processing	Each image has height, width, and color channels. Batch adds one more axis.
5D Tensor	Stack of 4D tensors.	Videos: `(N,F,H,W,C)`	5 axes	`(batch, frames, height, width, channels)`	Video models, action recognition	A video is a series of images (frames), so you add a frame axis.

==========

Quick Concept Summary

- Higher rank = more axes.
- Shape tells how many elements are stored along each axis.
- Images add the *channel* dimension.
- Videos add the *frame* dimension.
- Batches add the *batch* dimension.

Rank = number of axes.
Scalar 0, vector 1, matrix 2, video batch 5.

Shape = elements per axis.
Example: (2,3) → 2 rows, 3 columns.

Size = multiply shape.
(2,3) → 6 items.

Quick Practical Summary

1D → row of values
2D → matrix or table
3D → sentences, time series batches
4D → image batches
5D → video batches

==========

Day 12: Virtual Environment

- Base environment contains global libraries.
- Virtual environment creates an isolated workspace.
- Each project has its own versions of libraries.
- Reduces conflicts, makes deployment stable on servers.

- Good practice for Django, Flask, ML, and production apps.
- Deep learning needs GPUs.
- Use Google Colab to run notebooks with GPU or TPU.
- Logistic Regression draws the best line that separates two classes.

==========

Day 14: Framing the Problem

1. Turn a business question into a prediction or classification task.
2. Decide if labels exist.
   Supervised = labels present.
   Unsupervised = no labels.
3. Check if an existing product or model solves the problem.
4. Gather data from internal or external sources.
5. Decide performance metric.
   Accuracy, F1, RMSE, MAE.
6. Choose training mode.
   Batch learning trains on full dataset.
   Online learning updates model with new data continuously.
7. Check assumptions.
   Data quality, distribution, size, balance, missing values.

Churn Rate

- Number of users leaving in a time period.
- Growth happens when new users > churned users.
- Decline happens when churned users > new users.

==========

Day 15: Data Gathering

1. Working with CSV/TSV

- Load CSV: `pd.read_csv("cars.csv")`
- Load from URL by passing the link directly.
- Load TSV: `pd.read_csv("cars.tsv", sep="\t")`
- Add missing column names: `names=[...]`
- Set index column: `index_col="car_id"`
- Skip header rows: `header=1`
- Use specific columns: `usecols=[...]`
- Skip useless rows: `skiprows=[...]`
- Load limited rows: `nrows=100`
- Fix encoding errors: `encoding="latin1"`
- Skip corrupted lines: `on_bad_lines="skip"`
- Force datatypes: `dtype={'price': float}`
- Parse dates automatically: `parse_dates=["date"]`
- Use converters for cleaning values: `converters={"col": func}`

- Mark custom missing values: `na_values=["-", "null"]`
- Handle large files using chunks: `chunksize=1000`
  Process each chunk inside a loop.

Day 16: Working with JSON / SQL

- Read JSON: `pd.read_json("file.json")`
- Read SQL:
  `pd.read_sql_query("SELECT * FROM table", connection)`

Day 17: Fetching data from APIs

apis are data pipeline that help two software to interact

we use **request** library in python that make an http request to hit any route/api

rapidapi for different kind of data apis

Day 18: Web scraping

==========

Day 19: Understanding gathered data

Asking basic questions:

- import pandas as pd
- df = pd.read_csv("train.csv")

1. **How big is the data?**
   df.shape → tells us the size of data

2. **How does the data look like?**
   df.head() → it preview us first 5 rows
   df.sample(5) → give us 5 random rows to avoid biasedness in data that lies in somewhere(much better approach)

3. **What is the data type of columns?**
   df.info() → tells us about datatypes

4. **Are there any missing values?**
   df.isnull().sum() → give us total no of missing values in each col
   Check percentage also:
   `(df.isnull().mean()*100)`
   Helps decide drop or fill.

5. **How does the data look mathematically?**
   df.describe() → gives us mathematical summary of all numerical columns
   Summary for numeric columns, mean, std, min, max.

6. **Are there any duplicate values?**
   `df.duplicated().sum()`
   Give us total duplicate rows.
   Remove duplicates: `df.drop_duplicates(inplace=True)`

7. **How is the correlation between columns?**
   `df.corr()`
   Pearson correlation for numerical columns.
   Positive means direct relation.
   Negative means inverse relation.

Additional Useful Checks
(Added to improve clarity and completeness)

8. **Unique Values**
   `df.nunique()`
   Helps detect categorical columns.
9. **Value Distribution**
   `df['col'].value_counts()`
   Good for class imbalance check.
10. **Outliers Check**
    `df.describe()` min vs max
    Large gaps hint outliers.
11. **Memory Usage**
    `df.memory_usage(deep=True)`
    Useful for large datasets.
12. **Column Names**
    `df.columns`
    Helps detect spaces or inconsistent naming.

==========

Data Types

- **Numerical:** age, salary, marks, height, weight
- **Categorical:** gender, country, state, caste, department

Exploratory Data Analysis (EDA)

Purpose of EDA

- Understand structure, patterns, and problems in the data.
- Detect missing values, outliers, imbalance, skewness.
- Guide feature engineering and model choice.

Day 20: Understanding gathered data

Exploratory Data Analysis (EDA)

Univariate Analysis

Single variable (column) analysis at a time.
Used to find frequency, distribution, outliers, skewness.

A. Categorical Data

Check frequency of categories.

1. **Countplot**
   `Import seaborn as sns -- sns.countplot(x='Survived', data=df)`
2. **Value counts**
   `df['Survived'].value_counts()`
3. **Bar chart**
   `df['Survived'].value_counts().plot(kind='bar')`
4. **Pie chart (percentage)**
   `df['Survived'].value_counts().plot(kind='pie', autopct='%.2f')`

Notes:

- Use countplots for comparison.
- Pie charts are useful only when categories are few.

B. Numerical Data

Continuous values like Age, Height, Salary.

1. Histogram

Turns continuous data into ranges (bins).
`plt.hist(df['Age'], bins=5)`

Good for distribution shape.

2. Displot

Improved histogram with probability density.
`sns.displot(df['Age'])`

Shows peaks, spread, skew.

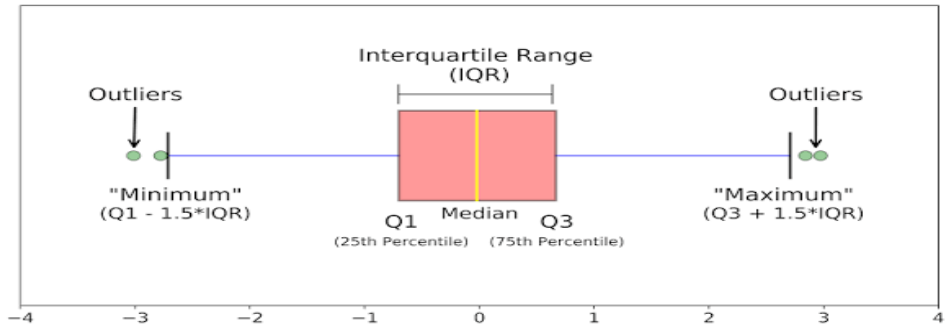
3. Boxplot

Gives **5-number summary**:

- Min
- Q1
- Median
- Q3
- Max
- Detects outliers (= points outside 1.5 IQR)

```
sns.boxplot(y=df['Age'])
```

Use for outlier removal.



Other Plots

- violinplot: shows distribution + density (rarely used)
- rugplot: shows raw data points (rarely used)

Basic Stats

- `df['Age'].min()`
- `df['Age'].max()`
- `df['Age'].mean()`
- `df['Age'].skew()` (positive skew = tail to right, negative = tail to left)

==========

Day 21: Understanding gathered data

Exploratory Data Analysis (EDA)

Bivariate Analysis

Two variables at a time.

EDA plots depend on **data type interaction**.

• Categorical vs Categorical → countplot, heatmap
• Numerical vs Numerical → scatterplot, line plot
• Categorical vs Numerical → bar plot, box plot, violin plot

Goal
• Check relationships
• Detect patterns
• Measure impact of one variable on another

---

1. Scatter Plot (Numerical vs Numerical)

Use when
• Both variables are continuous
• Relationship strength matters

Example
```
sns.scatterplot(x=total_bill, y=tip, hue=sex, style=smoker, size=size)
```

Why use
• Shows correlation trend
• Detects clusters
• Detects outliers
• Hue and style show category impact

Special case
• Use line plot instead when x-axis is time

---

2. Bar Plot (Numerical vs Categorical)

Use when
• Numerical value depends on categories
• Comparison between groups

Example
```
sns.barplot(x=Pclass, y=Fare, hue=Sex)
```

Why use
• Shows average or aggregate value
• Easy comparison across categories

Note
• Bars represent mean by default
• Not suitable for distribution shape

---

3. Box Plot (Numerical vs Categorical)

Use when
• Distribution matters
• Outliers matter

Example
```
sns.boxplot(x=Sex, y=Age)
```

Why use
• Shows median and spread
• Detects outliers
• Compares distributions across groups

Best for
• Age, income, price comparisons

---

4. Distplot (Numerical)

Use when
• Distribution shape matters

Example
```
sns.distplot(df['Age'])
```

Why use
• Shows skewness
• Shows peaks
• Shows spread

Note
• Histogram + density curve
• Use before transformation

---

5. Heatmap (Categorical vs Categorical)

Use when
• Comparing two categorical variables
• Pattern visibility matters

Example
```
sns.heatmap(pd.crosstab(sex, survived))
```

Why use
• Shows intensity pattern
• Faster comparison than tables

Limitation
• Percentages not shown directly

Fix
```
titanic.groupby('sex').mean()['Survived'] * 100
```

---

6. Clustermap (Categorical vs Categorical)

Use when
• Behavior similarity matters

Example
```
sns.clustermap(pd.crosstab(SibSp, Survived))
```

Why use
• Groups similar rows and columns
• Shows hidden structure

Best for
• Behavioral analysis
• Pattern grouping

---

7. Pairplot (Multiple Numerical values)

Use when
• Many numerical variables exist
• Quick overview needed

Example
```
sns.pairplot(df)
```

Why use
• Scatter of every numeric pair
• Diagonal shows distribution
• Best bird-eye view

Limitation
• Slow for large datasets

---

8. Line Plot (Numerical vs Numerical)

Use when
• X-axis represents time
• Trend over time matters

Example
```
sns.lineplot(x=date, y=sales)
```

Why use
• Shows trend

• Shows seasonality
• Better than scatter for time data

Quick Decision Rule

• Relationship strength → Scatter
• Time trend → Line
• Group comparison → Bar
• Distribution + outliers → Box
• Category frequency → Countplot
• Many numerics → Pairplot
• Category interaction → Heatmap
• Similar behavior → Clustermap

Extra Important Points (Added for Better Understanding)

- Numerical data has distribution shape (normal, skewed)
- Skewed data may need log/box-cox transformation.
- Outliers can distort mean, correlation, and model accuracy.
- Always check value imbalance in categorical columns.
- Use `describe(include='all')` to summarize both numeric and categorical data.
- Matplotlib is older lib for visualization while the seaborn is modern one based on matplotlib.

==========

Day 22: Panda's profiling tool

Purpose
Automates Exploratory Data Analysis.
Generates a full HTML report from a DataFrame.
Saves time compared to manual EDA.

What the report shows
• Dataset overview
• Number of rows and columns
• Data types
• Missing values count and percentage
• Duplicates
• Descriptive statistics
• Correlation matrix
• Distribution of each column
• Warnings about data quality issues

Used when
• Quick understanding of a new dataset

• Initial EDA before modeling
• Checking data quality problems early

import pandas as pd
df = pd.read_csv("train.csv")

!pip install ydata_profiling
from ydata_profiling import ProfileReport
prof = ProfileReport(df)
prof.to_file(output_file="output.html")

Profiling saves time.
Manual EDA builds intuition.

Best Practice

Use profiling first.
Then do manual EDA.
Combine both for best results.

==========

Day 23: Feature Engineering (IMP)

Feature engineering is the process of creating, modifying, and selecting input features using domain knowledge so ML models learn patterns better and give higher accuracy.

Purpose:
• Improve model performance
• Reduce noise
• Make patterns easier for algorithms to learn
• Reduce overfitting and training time

DATA PREPROCESSING VS FEATURE ENGINEERING

Data Preprocessing:
• Works on raw data
• Focuses on data quality (accuracy, completeness, consistency)
• Makes data usable
• Steps include cleaning, formatting, and fixing errors

Examples:
• Handling missing values, Removing duplicate, Fixing inconsistent data, Correcting data types

Feature Engineering:
• Works after basic preprocessing
• Focuses on features, not raw data

• Shapes data for better learning
• Strongly depends on domain knowledge

1. # FEATURE TRANSFORMATION

Modify existing features so algorithms perform better.

Common techniques:

a. Missing Value Imputation
• Mean, median, mode
• KNN imputation
• Model based imputation

b. Handling Categorical Variables
• Label Encoding
• One Hot Encoding
• Target Encoding

c. Outlier Handling
• Detection using IQR, Z score
• Capping
• Removal when justified

d. Feature Scaling
• Standardization
• Normalization
• Min Max scaling

Used for:
• Distance based models
• Gradient descent based models

2. # FEATURE CONSTRUCTION

Create new features from existing ones.

Techniques:
Feature splitting
Example: Date → day, month, year
Feature grouping
Example: Age → age groups
Mathematical combinations
Example: length × width = area

Goal:
• Capture hidden patterns
• Add meaningful information

3. **FEATURE SELECTION**
Select only important features for the model.

Why needed:
Reduce overfitting, Improve model speed, Remove irrelevant and redundant features

Methods:
Filter methods (correlation, chi square)
Wrapper methods (forward, backward selection)
Embedded methods (Lasso, decision trees)

4. **FEATURE EXTRACTION**
Automatically transform data into new features, usually reducing dimensions.

Key idea: Original features are replaced with new ones

Examples: **PCA, LDA, t SNE**

Real world example:
• Instead of number of rooms and washrooms
• Use total area in square feet

Used when:
• Data has high dimensions
• Visualization is needed
• Noise reduction is required

==========

Day 24: FEATURE SCALING (FEATURE TRANSFORMATION)

Feature scaling is a technique used before training ML models to bring all independent features to a similar scale or range.

Why feature scaling is needed:
• Some features have large values and dominate others
• Distance based algorithms get biased
• Gradient descent becomes slow or unstable
• Model accuracy drops without scaling

Required for:
• Distance based algorithms
• Gradient descent based algorithms

Not required for:
• Tree based models

ALGORITHMS THAT REQUIRE FEATURE SCALING

Scaling required:
K Means clustering
K Nearest Neighbors
Principal Component Analysis (PCA)
Artificial Neural Networks
Linear Regression with Gradient Descent
Logistic Regression
Support Vector Machine

Scaling not required:
Decision Tree
Random Forest
Gradient Boosting
XGBoost

Reason:
Tree models split data based on conditions, not distance.

TYPES OF FEATURE SCALING

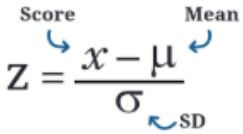
1. **STANDARDIZATION(Default choice)**
   Also called Z Score Normalization.

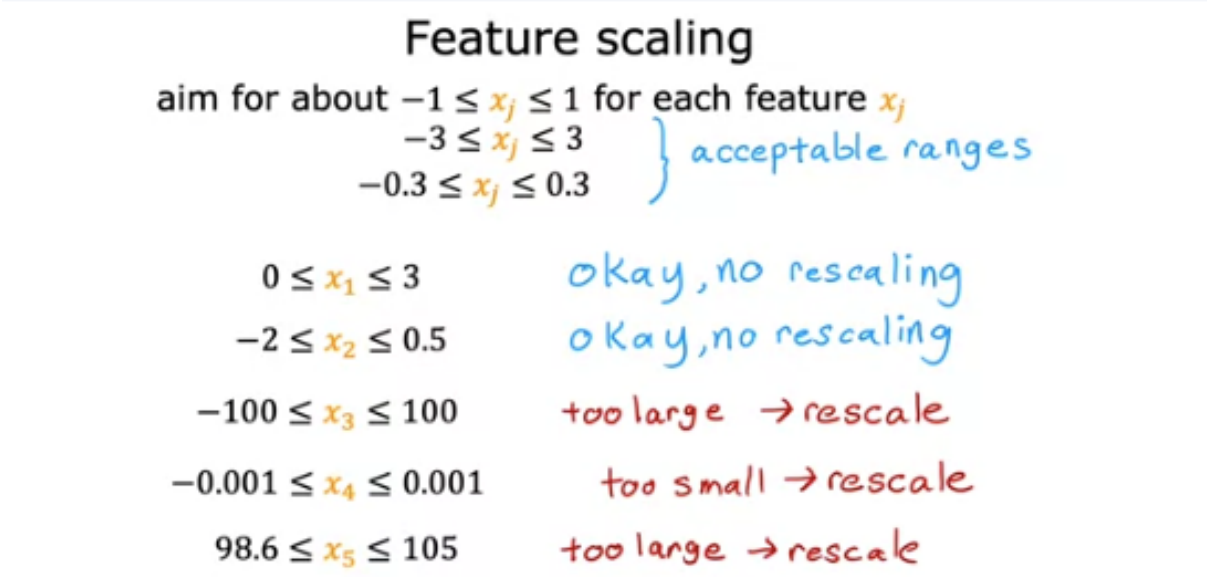
Definition: Transforms data so mean becomes 0 and standard deviation becomes 1.

Formula: z = (x − mean) / standard deviation

Properties:
• No fixed range and handles negative values
• Suitable when min and max values are **unknown**

When to use:
• Data follows normal distribution
• Outliers exist but are not extreme
• Most ML models use this one





==========

Day 25: FEATURE SCALING

2. NORMALIZATION

- Scales data into a fixed range.Use when min and max values are known
- **General range:** 0 to 1 or −1 to 1

Types of Normalization:

a. Min Max Scaling(use when bounds are known)

Most commonly used.

Formula: (x − min) / (max − min)

Properties:
• Values fall between 0 and 1
• Sensitive to outliers

$$X' = \frac{X - Xmin}{Xmax - Xmin}$$

---

b. Mean Normalization

Formula: (x − mean) / (max − min)

Properties:
• Data centered around 0
• Rarely used in practice

Reason:
Standardization performs better in most cases.

$$x' = \frac{x - \mu}{\max(x) - \min(x)}$$

---

c. Max Absolute Scaling

Formula: x / max(|x|)

Properties: Range −1 to 1
• Preserves zeros

When to use:
• **Sparse data** (having more zero values)

$$X_{new} = \frac{X_i}{|X_{max}|}$$

---

d. Robust Scaling

Uses: Median And Interquartile Range (IQR)

Formula: (x − median) / IQR

When to use:
• **Data contains many outliers**
• Outliers are important and cannot be removed

$$X_{new} = \frac{X - X_{median}}{IQR}$$

NORMALIZATION VS STANDARDIZATION

Standardization:
• Mean becomes 0 and Standard deviation becomes 1
• No fixed range
• Works when min and max are unknown
• Performs best in most ML models

Normalization:
• Fixed range
• Requires min and max
• Sensitive to outliers

WHICH TO USE

Default choice → Standardization
Known bounds → Min Max
Many outliers → Robust
Sparse data → Max Absolute

Extra

~~Decimal Scaling Normalization~~

~~**Purpose:**~~
~~To move the decimal point of values until they fall within a small range, typically **(-1, 1)**.~~

~~**Formula:**~~
~~`x' = x / 10^j`~~
~~Where **j** is the number of digits in the largest absolute value.~~

~~**Example:**~~
~~Data: [12, 100, 500]~~
~~Largest value = 500 → 3 digits → j = 3~~

~~`x' = 12/1000 = 0.012, 100/1000 = 0.1, 500/1000 = 0.5`~~

~~Normalized data → [0.012, 0.1, 0.5]~~

~~**When to use:**~~
~~When you want a **quick, simple normalization** and your data has **consistent scale** (no huge outliers~~).

~~**DISCRETIZATION**~~

~~Discretization converts continuous numerical data into discrete categories or bins.~~

~~Examples:~~
~~• Age → 0–10, 11–20, 21–30~~
~~• Age → young, adult, old~~

~~Why used:~~
~~• Simplify data~~
~~• Improve interpretability~~
~~• Useful for rule based models~~

Types:
• Equal-width binning
• Equal-frequency binning

StandardScaler and fit_transform Explained

Purpose

- Standardize numeric features so they have **mean = 0** and **std = 1**.
- Makes all features **comparable** for ML models (Logistic Regression, SVM, etc.).

Line we are studying

```
X_train_scaled = scaler.fit_transform(X_train)
```

Step-by-step working

1. **Fit (compute mean and std)**
    - `fit` calculates **mean** and **standard deviation** of each numeric column in training data.
    - Saves internally in the scaler:
        - `scaler.mean_` → column-wise mean
        - `scaler.scale_` → column-wise std

Example:
Column `Age = [20, 30, 40]`

```
mean = (20+30+40)/3 = 30
std = sqrt(((20-30)^2 + (30-30)^2 + (40-30)^2)/3) ≈ 8.165
```

---

2. **Transform (scale data)**
    - For each value:

```
3. X_scaled = (X - mean) / std
```

    - Using the previous example:

```
4. 20 → (20-30)/8.165 ≈ -1.224
5. 30 → 0
6. 40 → 1.224
```

- **Result**: a new array where each column has **mean=0, std=1**.

---

3. **fit_transform**

- Shortcut that does **fit + transform together** on training data.
- Saves mean/std internally for later use on test data.

---

4. **Transform test data**

```
X_test_scaled = scaler.transform(X_test)
```

- **Do not fit again on test**
- Use **mean/std from train** to scale test data
- Prevents **data leakage** (ensures consistency).

Why this is important

- Without scaling, features like `Fare` (0–500) would dominate smaller features like `SibSp` (0–8).
- Standardization helps the model **treat all features equally**.]

Summary:

- `fit_transform` → compute mean/std from train, scale train data
- `transform` → scale test data using train statistics
- Keeps train/test **on the same scale**, avoids **data leakage**

==========

Day 26: Encoding Categorical Variables

Categorical data is a part of feature transformation. It contains values in string form and must be converted into numbers before feeding data to ML models.
Categorical data is mainly of two types: NOMINAL and ORDINAL Choosing the correct encoding method depends on this distinction.

NOMINAL CATEGORICAL DATA: Nominal data has categories with no natural order or relationship among them. The categories are independent of each other.

Examples:
Color → Red, Blue, Green
City → Lahore, Karachi, Islamabad

There is no meaning in saying one category is greater or smaller than another. For such data, ordinal or label based numbering must not be used because it introduces false relationships.

ORDINAL CATEGORICAL DATA: Ordinal data has categories with a meaningful order or ranking. Here, the order carries information, so encoding must preserve this ranking.

Examples:
Grades → A, B, C
Reviews → Good, Average, Bad
Education → High School, Bachelor, Master

ORDINAL ENCODING

Ordinal Encoding is used for ordinal categorical input features. Each category is assigned a numerical value according to its rank or order.

Example: Bad → 0 Average → 1 Good → 2

from sklearn. preprocessing import OrdinalEncoder
oe = OrdinalEncoder(categories=[["Bad", "Average", "Good"]])
Important note: The order must be defined correctly. A wrong order misleads the model.

LABEL ENCODING

Label Encoding is similar to ordinal encoding but is mainly used for encoding output labels, not input features. It converts class labels into numeric form.

Example: Spam → 1 Not Spam → 0

- from sklearn.preprocessing import LabelEncoder
- le = LabelEncoder()
- y_encoded = le.fit_transform(y)

ORDINAL ENCODING VS LABEL ENCODING: Ordinal Encoding is used for input features with known order. Label Encoding is used for output labels or target variables. Using label encoding on nominal input features is a common mistake because it introduces false ordering.

==========

Day 27: Encoding Categorical Variables

One Hot Encoding

One Hot Encoding is used to handle **NOMINAL CATEGORICAL DATA where categories have no natural order.** It converts each category into a separate binary column, forming a vector representation of the original feature. This is the most commonly used encoding technique in machine learning. ML algorithms work with numerical data, not strings. OHE avoids assigning artificial order to categories, which would mislead models.

Example:
Color column with values Red, Blue, Green becomes three columns: Color_Red, Color_Blue, Color_Green. Each row contains a 1 for the present category and 0 for others. As the number of categories increases, the number of generated columns also increases.

Original data:

Sample	Color
1	Red
2	Blue
3	Green
4	Red

After one hot encoding:

Sample	Color_Red	Color_Blue	Color_Green
1	1	0	0
2	0	1	0
3	0	0	1
4	1	0	0

We changed string into kind of vector

HIGH CARDINALITY ISSUE

When a feature has many unique categories, OHE creates too many columns. This increases memory usage, training time, and risk of overfitting. This is known as the curse of dimensionality.

In ML, its general wisdom that input column should not have any relationship among them or let say dependent over each other.

DUMMY VARIABLE TRAP AND MULTICOLLINEARITY

Dummy variable trap occurs when all one hot encoded columns are used together. Since the sum of all dummy columns equals 1, one column becomes redundant and creates perfect multicollinearity.

Solution: Drop one dummy column. No information is lost because rows with all zeros implicitly represent the dropped category.

Example: If Red, Blue, Green exist, drop Green.
All zeros → Green.

HANDLING MOST FREQUENT CATEGORIES

When a feature has many categories, keep only the most frequent ones and group the rest into a single category called "Other". This reduces dimensionality and noise.

Effect on model:
Rare categories often carry little predictive power. Grouping them improves generalization. Some fine detail is lost, but overall model stability improves.

from sklearn. preprocessing import OneHotEncoder

ohe = OneHotEncoder (handle_unknown="ignore", drop="first")

Converts categories into binary vectors, drops one column to avoid multicollinearity, and ignores unseen categories during testing.

==========

Day 28: Column Transformer

In real datasets, different columns require different preprocessing steps. Numerical columns need imputation or scaling. Categorical columns need encoding. Text or binary columns need separate handling. Applying transformations column by column using X_train[["age"]] creates multiple NumPy arrays which must be merged later. This process becomes messy, error prone, and hard to reproduce.

ColumnTransformer from scikit-learn solves this problem by applying different transformations to different columns in a single step and returning one combined output array. It keeps column order consistent and ensures the same transformations apply to training and test data.

HOW COLUMN TRANSFORMER WORKS

ColumnTransformer takes a list of transformers. Each transformer specifies a name, a transformation object, and the columns it applies to. Columns not listed can either be dropped or passed unchanged using remainder="passthrough".

Example idea:
Age column gets imputed.
Education column gets ordinal encoded.
City column gets one hot encoded.
Remaining columns pass as is.

- **from sklearn.compose import ColumnTransformer**
- from sklearn.impute import SimpleImputer
- from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder

ct = ColumnTransformer (
transformers= [
("num", SimpleImputer(strategy="mean"), ["age"]),
("ord", OrdinalEncoder (), ["education"]),
("cat", OneHotEncoder(handle_unknown="ignore"), ["city"])
],
remainder="passthrough")
ColumnTransformer is often used inside a Pipeline to chain preprocessing and model training into one object. This avoids data leakage and simplifies deployment.

==========

Day 29: Pipeline Transformer

In ML, you usually do these steps:

- Fill missing values, Scale numbers, Encode categories, Train model
- You must do **same steps** on: Training data, Test data, New unseen data
- If you forget or change order, model breaks.

==========

Example WITHOUT Pipeline

Suppose dataset has:
• Age has missing values
• Salary has large scale difference

Steps done manually.

```
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

# 1. Handle missing values
imputer = SimpleImputer(strategy="mean")
X_train = imputer.fit_transform(X_train)
X_test = imputer.transform(X_test)

# 2. Scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# 3. Train model
model = LogisticRegression()
model.fit(X_train, y_train)

# 4. Predict
y_pred = model.predict(X_test)
```

Problems here:

• Too much code
• Easy to forget transform on test data
• Order must be remembered
• Hard to reuse and Risk of data leakage

==========

What is Pipeline

- Pipeline connects steps in sequence. Output of one step becomes input of next step.
- Fit once. Transform automatically.
- Same preprocessing for train, test, new data.
- Pipeline Structure = Preprocessing → Preprocessing → Model

==========

Example WITH Pipeline

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="mean")),
    ("scaler", StandardScaler()),
    ("model", LogisticRegression())
])

pipe.fit(X_train, y_train)
y_pred = pipe.predict(X_test)
```

What happened internally:

• Missing values handled, Data got scaled, Model trained, Correct order guaranteed

==========

What is a Transformer

Transformer:
• Learns from data using fit
• Changes data using transform

Examples:
• SimpleImputer
• StandardScaler
• OneHotEncoder

Rule:
• All steps except last must be transformers
• Last step must be estimator (model)

==========

Why Pipeline is Important

• Prevents data leakage
• Same logic for train and test
• Clean and readable code
• Easy cross-validation
• Easy deployment

Without pipeline:
You repeat steps.

With pipeline:
One object controls everything.

==========

Pipeline with Cross Validation

Without pipeline:
Preprocessing leaks test data.

With pipeline:
Each fold does preprocessing correctly.

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(pipe, X, y, cv=5)
```

Safe and correct.

==========

Beginner Mental Model

Think of pipeline as: Assembly line. Raw data enters. Clean data comes out. And Model predicts. You never touch intermediate steps.

==========

Day 30: Function Transformer

A normal distribution of data (Gaussian distribution) is characterized by a symmetric, bell-shaped curve where data clusters around a central mean, with frequency decreasing further from the center. The "opposite" of this is skewed or non-normal data, where the distribution is lopsided, featuring a long tail on one side.

Some ML algorithms perform better when features follow a normal distribution. Function Transformer helps convert skewed data into a more normal shape.Applies a mathematical function to a column.Why normal distribution matters

- Many models assume normality.
- It improves distance-based models like KNN and SVM.
- It stabilizes variance.
- It makes relationships easier to learn.

Common Transformations

1. **Log Transform**
   What it does

- Compresses large values. Reduces right skew.

When to use

- Right-skewed data.
- Values must be positive.

Formula
$y = \log(x)$

Python
```
df["col"] = np.log(df["col"])
```

Note

- Cannot apply to zero or negative values.
- Use log1p(x) for zeros.

---

2. **Reciprocal Transform**
   What it does

- Large values become small.
- Small values become large.

When to use

- Strong right skew.
- When large values dominate.

Formula
$y = 1 / x$

Python
```
df["col"] = 1 / df["col"]
```

---

3. **Square Transform**
   What it does

- Stretches large values.
- Reduces left skew.

When to use

- Left-skewed data.

Formula
$y = x^2$

Python
```
df["col"] = df["col"] ** 2
```

---

4. **Square Root Transform**
   What it does

- Reduces right skew.
- Less aggressive than log.

When to use

- Right-skewed data.
- Works with zero values.

Formula
$y = \sqrt{x}$

Python
df["col"] = np.sqrt(df["col"])

---

5. Custom Transformer
   What it does

- Applies your own function.

When to use

- When built-in transforms do not work well.

Python
df["col"] = df["col"].apply(lambda x: x ** 0.3)

How to Check if Data is Normal

1. Histogram
   sns.displot(df["col"])
2. Skewness Value
   df["col"].skew()

- 0 means normal.
- Positive means right-skew.
- Negative means left-skew.

3. Q-Q Plot (Most reliable)
   from scipy import stats
   stats.probplot(df["col"], plot=plt)

==========

Day 31: Power Transformer

Automatically find the best power to apply to a feature to make it close to normal.

Difference from Function Transformer

- Function Transformer uses fixed formulas.
- Power Transformer finds the best power automatically.

Types of Power Transformer

1. **Box-Cox Transform**
   What it does

- Finds best power λ for transformation.

Limitation

- Only works on positive values.

Python
```
from sklearn.preprocessing import PowerTransformer
pt = PowerTransformer(method="box-cox")
df_transformed = pt.fit_transform(df[["col"]])
```

---

2. **Yeo-Johnson Transform**
   What it does

- Same idea as Box-Cox.
- Supports zero and negative values.

Why better

- Works on all numeric data.
- Often gives better results.

Python
```
pt = PowerTransformer(method="yeo-johnson")
df_transformed = pt.fit_transform(df[["col"]])
```

How Power Transformer Works

- It tries different powers of x.
- It selects the power that makes data most normal.
- You do not choose the power manually.

When to Use Which

Use Function Transformer when

- You understand your data well.
- You want full control.
- You know which formula fits your data.

Use Power Transformer when

- You want automation.
- You have many features.
- You want consistent normalization.

==========

DAY 32: BINNING AND BINARIZATION

Binning converts numerical values into categories.It is also called discretization.

Why convert numbers into categories
Numbers are precise. Categories add structure.
You bin to:

- Reduce noise.
- Handle outliers.
- Improve model stability.
- Make patterns easier to learn.
- Improve interpretability.

Example
Downloads:

- 23 → Low
- 3,000 → Medium
- 3,000,000 → High

Instead of raw values, you use categories.

WHY BINNING HELPS WITH OUTLIERS

Example:
Salaries: 20k, 25k, 30k, 1,000,000
The last value skews the scale.
Binning groups it into "High income" and reduces impact.

TYPES OF BINNING

1. **UNSUPERVISED BINNING**
   No target variable involved.

a) Equal width binning (Uniform)

- Divide range into equal-sized intervals.

Example:
Values: 0 to 100
Bins: 5
Bin width = 20
Bins:

- 0–20
- 21–40
- 41–60
- 61–80
- 81–100

b) Equal frequency binning (Quantile)

- Each bin has same number of samples.

Example:
Scores: [10, 20, 30, 40, 50, 60, 70, 80]
Bins: 4
Each bin has 2 values:

- [10, 20]
- [30, 40]
- [50, 60]
- [70, 80]

c) K-means binning

- Uses clustering to group similar values.
- Centers define bins.

Example:
Ages: [18, 19, 20, 35, 36, 37, 60, 61, 62]
K=3 clusters:

- Young: 18–20
- Middle: 35–37
- Senior: 60–62

---

2. **SUPERVISED BINNING**
   Uses target variable.

a) Decision tree binning

- Tree finds splits that best separate target classes.
- Each leaf becomes a bin.

Example:
Loan amount predicts default.
Tree splits:

- ≤ 50k → Low risk
- 50k–200k → Medium risk
    - 200k → High risk

---

3. **CUSTOM BINNING**
   You define bins using domain knowledge.

Example:
Age groups:

- 0–12 → Child
- 13–19 → Teen
- 20–35 → Young adult
- 36–60 → Adult
- 60+ → Senior

ENCODING DISCRETIZED VARIABLES

After binning, you encode categories.

Common encodings:

- Ordinal encoding: Low=0, Medium=1, High=2
- One-hot encoding: Low=[1,0,0], Medium=[0,1,0], High=[0,0,1]

SKLEARN: KBinsDiscretizer

You provide:

- n_bins: number of bins

- strategy: "uniform", "quantile", or "kmeans"
- encode: "ordinal" or "onehot"

Example concept:
Convert income into 3 bins using quantile strategy and ordinal encoding.

BINARIZATION

What is binarization
Convert values into 0 or 1 based on a threshold.

Why use it

- For presence or absence.
- For yes or no decisions.
- For rule-based features.

Example:
Score ≥ 50 → 1
Score < 50 → 0

Sklearn:
Use Binarizer class.
You set a threshold.
Values above threshold → 1.
Values below or equal → 0.

==========

DAY 33: HANDLING MIXED DATA

What is mixed data
One column contains both letters and numbers.

Example:
Cabin column:

- B5
- D1
- A8

Problem
Model cannot learn from mixed types directly.

Solution
Split into two columns:

- Cabin_Letter: B, D, A
- Cabin_Number: 5, 1, 8

Then:

- Encode letters.
- Keep numbers as numeric.

==========

DAY 34: HANDLING DATE AND TIME VARIABLES

Dates contain hidden information.
You extract features from them.

Example:
Date: 2026-01-28 14:45

Extract:

- Year: 2026
- Month: 1
- Day: 28
- Day of week: Wednesday
- Hour: 14
- Is weekend: No
- Is month end: Yes or No

EXPENSE TRACKER ML EXAMPLE

Raw data:

- Amount
- Date
- Time
- Category

Feature engineering:
From Date:

- Day of week
- Month
- Is weekend

From Time:

- Hour
- Morning, afternoon, evening, night

From Amount:

- Binned spending level: Low, Medium, High
- Binary: High expense or not

Use cases:

- Predict next expense category.
- Detect unusual spending.
- Forecast monthly spending.

==========

DAY 35: HANDLING MISSING VALUE

Removing values

Removing: will discuss this today
Imputing:

- Univariate**: fill with** mean/median**,** random**,** end of distribution values **(**SimpleImputer)
- Multivariate**: KNN imputer, iterative imputer**

Complete Case Analysis, CCA: You remove rows where any column has missing values.

Example: 1000 rows - 50 rows have missing Age - You delete those 50 rows - You train on 950 rows

Assumption:
(MCAR) Missing Completely At Random: Missing values are random. Not linked to any feature or target.

When to use CCA

- Missing data less than **5 percent**
- Data is MCAR
- Dataset is large
- Removing rows does not reduce statistical power

Problem

- Lose data
- If not MCAR, you introduce bias

Mean only uses rows where all values exist.

Rule
Numerical data

- Less than 5 percent missing → CCA or mean or median

Categorical data

- Replace with mode

==========

DAY 36: HANDLING NUMERICAL MISSING VALUE (Imputing)

Univariate imputing: Univariate means you use only that column to fill its missing values.

4 techniques

1. **Mean Imputation**
   Use when data is normally distributed.

Effect

- Distribution becomes tighter
- Variance decreases
- Weakens relationship with other features
- May introduce artificial central clustering

Example
Age mean = 30
All missing replaced with 30

Check variance before and after
Variance shrinks after mean imputation.

2. **Median Imputation**
   Use when data is skewed.

Example: Income data is right skewed -> Use median instead of mean

More robust to outliers.

3. **Arbitrary Value Imputation**
   Replace missing with a fixed value.

Example
Age missing → fill with 999
Used mostly for categorical or special flagging.

4. End of Distribution Imputation

Used when missing may indicate extreme values.

If normal distribution
Replace with
Mean + 3 × standard deviation
or
Mean − 3 × standard deviation

If skewed distribution
Use IQR rule

IQR = Q3 − Q1

Upper bound = Q3 + 1.5 × IQR
Lower bound = Q1 − 1.5 × IQR

You push missing values to extreme range.

Use when

- Missing values may represent abnormal cases
- You want model to detect missing as extreme behavior

==========

DAY 37: HANDLING CATEGORICAL MISSING VALUE

Two techniques:

1. **Most Frequent Replacement**
   Replace missing with mode.

Example: City column - Most common value is Lahore - Fill missing with Lahore

Use when

- Missing is random
- Small percentage missing

2. **New Category Method**

Create new label
Example
Gender → Male, Female, Missing

Use when

- Missing has meaning
- MNAR possible
- Missing percentage is high

Often improves tree models.

DAY 38: HANDLING MISSING VALUE

Random Sample Imputation:

You randomly pick a value from existing non-missing values. It work for both numerical and categorical data

Example: Age values: 20, 25, 30, 40 → Missing row gets randomly one of these.

Advantages

- Preserves distribution - Preserves variance - Good for linear models

Disadvantage

- Need stored distribution during deployment
- More memory usage

Missing indicator:

You add a new binary column.

Example
Age_missing
1 if Age was missing
0 if not

Useful when

- Missing itself carries information
- Often improves accuracy

SimpleImputer add_indicator=True adds this automatically.

Automatic selection of parameter:

Grid Search CV for Best Imputation

You treat imputation strategy as hyperparameter.

Example
Test
Mean
Median
Most frequent
KNN

Model chooses strategy with best cross-validation score.

==========

DAY 39: HANDLING MISSING VALUE (MULTIVARIATE)

Univariate ignores other columns.
Multivariate uses other features to estimate missing values.

Two techniques:

KNN IMPUTER:

uses K Nearest Neighbors.

How it works: Example dataset

Age Salary
25 50k
30. 60k
35 80k
NaN 55k

To fill missing Age

Step 1
Compute distance between missing row and other rows using Salary.

Step 2
Find K nearest rows based on Euclidean distance.

Step 3
Take average Age of nearest neighbors.

If K = 2
Closest Salary rows: 50k and 60k
Their Ages: 25 and 30
Imputed Age = 27.5

Advantages

- More accurate - Uses structure in data

Disadvantages

- Slow , Heavy computation
- Needs full dataset at prediction time, Memory expensive

==========

DAY 40: HANDLING MISSING VALUE (MULTIVARIATE)

Iterative imputer - MICE:

MICE means Multivariate Imputation by Chained Equations.

Idea: Each column with missing values is predicted using other columns as predictors.

Missing Types

MCAR: Missing completely random. No pattern.

MAR: Missing depends on other observed variables.

- Example: Salary missing depends on Education level.

MNAR: Missing depends on unobserved variable or the value itself.

- Example: High earners hide salary.

How Iterative Imputer Works

Example

Columns
Age
Salary
Experience

- **Step 1**
  Initialize missing values with simple method like mean.

- **Step 2**
  Pick one column with missing values.
  Suppose Age.
- **Step 3**
  Use other columns Salary and Experience to train regression model predicting Age.
- **Step 4**
  Predict missing Age values.
- **Step 5**
  Move to next column with missing values.
  Repeat same process.
- **Step 6**
  Repeat full cycle multiple times until values stabilize.

This creates chained equations.

Advantages

- More accurate
- Uses relationships between features

Disadvantages

- Slow
- High memory usage
- Complex
- Risk of overfitting

~~EXTRA: HOW TO~~ CHOOSE BEST METHOD

Step 1: Check missing percentage

- Less than 5 percent → CCA or simple imputation
- 5 to 20 percent → median or mode or random sample
- More than 30 percent → consider dropping feature

Step 2: Check missing mechanism
MCAR → CCA or simple works
MAR → Multivariate works better
MNAR → Add missing indicator or new category

Step 3: Check distribution
Normal → Mean
Skewed → Median
Extreme meaning → End of distribution

Step 4: Check model type

Linear models

- Sensitive to variance
- Prefer random sample or multivariate

Tree models

- Handle missing better
- New category works well

Step 5: Check computational limits
Low memory → Mean or median
High memory allowed → KNN or Iterative

Step 6: Validate with cross validation
Use GridSearchCV
Compare model score

Always measure

- Accuracy
- Variance change
- Feature correlation change

Core Principle
Never choose method blindly.
Understand

- Why data is missing
- How much data is missing
- What model you are using
- What resources you have

MODEL TYPE MATTERS

Linear Regression or Logistic Regression
Sensitive to variance shrinkage.
Prefer random sample or multivariate.

Tree Models like Random Forest
Handle imputation noise better.
Median works fine.

==========

DAY 41: HANDLING OUTLIERS

Introduction

- Outliers are extreme values that lie far from most observations.
- Example: Age column mostly between 18 and 70 - One record shows age = 300
- In data cleaning, you often remove outliers
- In anomaly detection or fraud detection, outliers are the target, so you keep them

Why outliers matter in ML

A. Algorithms highly affected by outliers

These algorithms calculate weights or minimize error using squared loss.

- Linear Regression: One extreme value increases error heavily, Line shifts toward outlier
- Logistic Regression: Uses gradient descent, Extreme values affect coefficient updates
- AdaBoost: Focuses on misclassified points, Outliers get high weight
- Deep Learning: Uses gradient descent, Large values create large gradients, Training becomes unstable

Reason: These models rely on distance or error magnitude.Outliers increase error significantly.

B. Algorithms less affected by outliers

- Decision Tree: Splits based on thresholds, One extreme value does not shift all splits
- Random Forest: Ensemble of trees, Robust to noise
- Gradient Boosting: Tree-based boosting, Less sensitive than linear models
- XGBoost: Regularized tree boosting, Handles outliers better

Reason: Tree models split data using rules, not distance from mean.

How to detect outliers

A. Normal Distribution: If data follows Gaussian distribution

- Use Z-score rule
- Z = (x − mean) / standard deviation
- If (Z > +3 or Z < −3) → Then value is considered an outlier

B. Skewed Distribution: Use IQR method

- IQR = Q3 − Q1
- Lower bound = Q1 − 1.5 × IQR
  Upper bound = Q3 + 1.5 × IQR
- If value < lower bound or value > upper bound → Then it is an outlier

C. Any Distribution

- Use percentile method
- Lower threshold = 2.5 percentile
- Upper threshold = 97.5 percentile
- Values outside these limits are treated as outliers.
- You choose percentile based on tolerance.

How to treat outliers

A. **Trimming:** Remove rows containing outliers.

Use when

- Outliers are errors
- Data size is large
- Extreme values are unrealistic

Risk: You lose information.

B. Capping or Winsorization: Replace extreme values with boundary values.

Example

- If upper limit = 100 then any value > 100 becomes 100

Use when

- Outliers are real but too extreme
- You want to reduce influence
- Good for linear models.

C. Treat outliers as missing

- Mark them as NaN
- Impute using mean, median, or model

Use when: Value is likely measurement error

D. Discretization

Convert continuous variable into bins.

Example
Income →
0–20k
20k–50k
50k+

Effect

- Reduces impact of extreme values
- Makes model less sensitive

Common in tree-based or logistic models.

Practical workflow

- Step 1: Understand business objective
- Step 2: Visualize distribution through Histogram or Boxplot
- Step 3: Detect using (Z-score for normal data) and (IQR for skewed data)
- Step 4: Decide treatment based on Model type, Domain knowledge, Data size

Techniques for outliers detection and removal:

- Z - Score treatment
- IQR based filtering

- Percentile
- Winsorization

==========

DAY 42: HANDLING OUTLIERS

Outliers Removal And Detection Using Z Score:

Z-score measures how many standard deviations a value is from the mean.

Formula: Z = (x − mean) / standard deviation

Empirical Rule for Normal Distribution

If data follows normal distribution

- 68 percent values lie within ±1 standard deviation
- 95 percent values lie within ±2 standard deviations
- 99.7 percent values lie within ±3 standard deviations

Outlier rule using Z-score: If Z > 3 or Z < −3 → treat as **outlier**

Important: Z-score method works properly only when data is approximately normal.

Finding Outlier Boundaries

- Upper limit = mean + 3 × standard deviation
- Lower limit = mean − 3 × standard deviation

Example for column cgpa

- Upper boundary: df['cgpa'].mean() + 3 * df['cgpa'].std()
- Lower boundary: df['cgpa'].mean() - 3 * df['cgpa'].std()

Detecting Outliers Using Boundaries: After calculating limits, find rows outside range

- df[(df['cgpa'] > upper_limit) | (df['cgpa'] < lower_limit)]
- These rows contain outliers.

Calculating Z-Score Column

- df['cgpa_zscore'] = (df['cgpa'] - df['cgpa'].mean()) / df['cgpa'].std()

Keeping Only Non-Outlier Rows: Filter rows where Z-score is between −3 and 3

- df[(df['cgpa_zscore'] < 3) & (df['cgpa_zscore'] > -3)]

Trimming: Remove rows where outliers exist.

- Use when: Outliers are data errors, Dataset size is large
- Risk: Loss of information

Capping: Instead of removing rows, replace extreme values with boundary values.

Logic

- If value > upper_limit → replace with upper_limit
- If value < lower_limit → replace with lower_limit
- Else → keep original value

Using np.where
Assign upper limit if above
Assign lower limit if below
Otherwise keep original value

==========

DAY 43: HANDLING OUTLIERS

Outliers Removal And Detection Using IQR Method:

When to use IQR: Use IQR method for skewed data.Do not use Z-score if distribution is not normal.
Visualize using boxplot:

```
sns.boxplot(x = df['cgpa'])
```

Checking skewness: `df['cgpa'].skew()`

Interpretation:

- Skew ≈ 0 → symmetric distribution
- Skew > 0 → right skewed, long right tail
- Skew < 0 → left skewed, long left tail

Large absolute value means stronger skewness.

Percentiles

- **25th percentile, Q1:** 25 percent values lie below this value.
- **75th percentile, Q3:** 75 percent values lie below this value.

IQR = Q3 − Q1
IQR measures spread of middle 50 percent data.

Outlier boundaries using IQR

Lower limit = Q1 − 1.5 × IQR
Upper limit = Q3 + 1.5 × IQR

Any value: < lower limit → outlier or > upper limit → outlier

Finding Q1 and Q3 in code

Q1 = df['cgpa'].quantile(0.25)
Q3 = df['cgpa'].quantile(0.75)

IQR = Q3 − Q1

Lower limit = Q1 − 1.5 * IQR
Upper limit = Q3 + 1.5 * IQR

Detecting outliers

df[(df['cgpa'] < lower_limit) | (df['cgpa'] > upper_limit)]

==========

DAY 44: HANDLING OUTLIERS

Outliers Removal and Detection Using Winsorization:

What is Winsorization: Winsorization means capping extreme values instead of removing them. You replace outliers with boundary percentile values. No rows are deleted.

Common practice among people

- 1st percentile as lower limit
- 99th percentile as upper limit

Sometimes 5th and 95th percentiles are used if data is highly noisy.

How it works

- Step 1: Find percentile limits
  - Lower limit = 1st percentile
    Upper limit = 99th percentile
- Step 2: Replace values
  - If value < lower limit → set equal to lower limit
  - If value > upper limit → set equal to upper limit
  - Else → keep original value

Why use Winsorization

- Prevent data loss
- Reduce effect of extreme values
- Stabilize linear and gradient-based models
- Useful for financial or income data

When to use

- When outliers are real but extreme

- When dataset is small
- When using linear regression, logistic regression, deep learning

Important difference

Trimming → removes rows
Winsorization → keeps rows, modifies values.

==========

DAY 45: FEATURE CONSTRUCTION | FEATURE SPLITTING

Types of Feature Engineering

1. **Feature transformation**
    - Missing value handling
    - Categorical encoding
    - Outlier handling
    - Feature scaling
2. **Feature construction**
    - Feature splitting
3. **Feature selection**
    - Will do it in last after covering algos
4. **Feature Extraction**

Earlier topics were feature transformation. Now focus on construction and splitting.

Feature Construction: Create new features from existing features using logic and domain knowledge.No fixed formula.Depends on understanding of data and problem.

Example from Titanic dataset: Columns

- SibSp, number of siblings or spouses
- Parch, number of parents or children

New feature: Family size = SibSp + Parch + 1

Add 1 to include the passenger.

Code: X['family_size'] = X['SibSp'] + X['Parch'] + 1

Why useful

- Single passengers behave differently from large families
- Model captures group behavior better
- Reduces noise from separate small columns

Feature Splitting: Break one feature into multiple meaningful parts. Used when a column contains combined information. Feature splitting extracts hidden structured information from raw text columns

Example from Titanic: Column
df['Name']

Name contains

- Title
- First name
- Last name

Extract title: Use split operation to separate string by comma and period.

Goal: Create new column such as: Mr, Miss, Mrs, Dr

==========

DAY 46: CURSE OF DIMENSIONALITY

Feature Selection: Will cover it in last, after covering algorithms because it is used after model building to check whether feature adding or removing affect model or not.

Dimension means number of features.
Adding features improves model only if they add useful information.
After a limit, performance stops improving or decreases.
Reason: noise increases, not signal.

In image classification, only pixels covering object matter. Extra background pixels add complexity, not accuracy. **More features ≠ more knowledge.**

Curse of dimensionality appears in high-dimensional data

- Images, many pixels
- Text data, many words
- Genomics, many genes
- The problem is sparsity among columns. In high dimensions, most values are zero.
- Example : 10,000 word features. Each document uses 100 words. 9,900 features are zero.
- Data points become far apart.Pattern learning becomes hard.

Problems

- Sparsity increases
- Distance measures lose meaning
- Overfitting risk increases
- Computation cost increases and training time increases
- Storage increases and performance may drop

Solution: Dimensionality Reduction

Reduce features while keeping important information.

Two methods

A. Feature Selection: Select important original features.

- **Forward Selection**: Start empty, Add useful features
- **Backward Elimination:** Start full, Remove useless features

Original features remain unchanged.

B. Feature Extraction
Create new reduced features from old ones.Transform data into lower dimension.

Techniques

- **PCA:** Maximizes variance
- **LDA:** Maximizes class separation
- **t-SNE:** Used for 2D or 3D visualization

==========

DAY 47: PRINCIPLE COMPONENT ANALYSIS (PCA)

Geometric Intuition

PCA is unsupervised machine learning problem technique, having just input no output
PCA which can transform higher dimension data into lower one while keeping the core essence of data

PCA Benefits: Data size reduction, result in faster execution of algo
Visualization in needed dimension

Feature selection is keeping only those columns that are important to find outputs
If we don't know which column is more necessary than we should project data on scatter plot and check their spread distance, which one having lengthy projection, select those columns which has bigger spread or variance.
If we come across such set of columns which has equal variance then we use feature extraction technique. Like rooms and washrooms are both important to predict price of any home, so we can't remove any column but instead we make a new column of house_size that will predict the price.
No of principle component would always be less than n (no of original features in data)

Why variance is important why not mean instead? formula is (x-x')**2/n
Spread is not variance but spread is directly proportional to variance

We must maximize variance as not to disturb relationship between data when transforming higher dimension data to lower dimension

EXTRA

1. Mean (center of data)

Mean = average of values

Formula
mean = (x1 + x2 + ... + xn) / n

Example

Data: [2, 4, 6]

Mean = (2 + 4 + 6) / 3 = 4

Intuition

Mean is the **center point** of your data.

Think:
You place weights on a rod. Mean is the balance point.

---

2. Variance (spread of data)

Variance tells how far values are from the mean.

Formula
variance = average of $(xi - mean)^2$

Example

Data: [2, 4, 6], mean = 4

Deviations:

- $(2 - 4)^2 = 4$
- $(4 - 4)^2 = 0$
- $(6 - 4)^2 = 4$

Variance = (4 + 0 + 4) / 3 = 2.67

Intuition

- Low variance → data is tight
- High variance → data is spread out

---

3. Standard Deviation (SD)

SD = square root of variance

Formula
SD = √variance

Example

SD = √2.67 ≈ 1.63

Why SD?

Variance is squared. Units become weird.
SD brings it back to original scale.

Intuition

Average distance from mean.

---

4. From 1D to 2D (important jump)

Now data has 2 features:

Example:
Height and Weight

Each point = (x, y)

Now we care about:

- Spread in x
- Spread in y
- Relationship between x and y

This leads to **covariance**

---

5. Covariance (relationship between features)

Covariance measures how two variables move together.

Formula
cov(x, y) = average of (xi − mean_x)(yi − mean_y)

Intuition

- Positive → both increase together
- Negative → one increases, other decreases
- Zero → no relation

---

6. Covariance Matrix

For 2D:

| var(x) cov(x,y) |
| cov(y,x) var(y) |

This matrix summarizes:

- spread
- direction

---

7. Eigenvectors and Eigenvalues

Now comes the core idea.

Given covariance matrix → we extract:

- Eigenvectors → directions
- Eigenvalues → importance of those directions

7.1 Eigenvectors (directions)

Eigenvector = direction in which data varies most(the direction where data points spread out the most)

Think:
Data cloud is tilted.

Eigenvector tells:
"this is the main direction of spread"

7.2 Eigenvalues (magnitude)

Eigenvalue tells:
"how much variance exists along that direction"

- Large eigenvalue → important direction
- Small eigenvalue → less important

Simple Analogy

Imagine a stretched ellipse:

- Long axis → major direction → high variance → big eigenvalue

- Short axis → minor direction → low variance → small eigenvalue

Eigenvectors = axes directions
Eigenvalues = axis lengths

---

8. Visual intuition

Data is like a tilted cloud.

Normal axes:

- x-axis
- y-axis

But data is not aligned with them.

Eigenvectors rotate axes to align with data.

---

9. Now connect everything to PCA

PCA = Principal Component Analysis

Goal:
Reduce dimensions(columns) but keep maximum information

Uses scikit-learn internally, but concept is pure math.

Step-by-step PCA flow

Step 1. Mean center data

Subtract mean from every point

Why?
We want data centered at origin

Step 2. Compute covariance matrix

Captures:

- spread
- relationships

Step 3. Compute eigenvectors and eigenvalues

From covariance matrix:

- Eigenvectors → directions of max variance
- Eigenvalues → how much variance

Step 4. Sort eigenvalues (descending)

Pick top k

Example:

- 2D → reduce to 1D → pick largest eigenvalue direction

Step 5. Project data

Project original data onto selected eigenvectors

This gives:

- reduced dimensions
- maximum preserved information

---

10. Final connection (everything together)

- Mean → centers data
- Variance → measures spread
- Covariance → shows relationships
- Covariance matrix → full structure
- Eigenvectors → best directions
- Eigenvalues → importance of directions
- PCA → selects top directions and projects data

---

11. One-line intuition

PCA finds the direction where your data spreads the most and keeps that, ignoring the rest.

---

12. Real-world intuition

Think of shadows:

- 3D object → 2D shadow
- You choose angle that preserves most shape

That angle = eigenvector
Importance = eigenvalue

==========

DAY 48: PRINCIPLE COMPONENT ANALYSIS (PCA)

Problem Formulation and code solution

PCA has to find such unit vector for which the value is miximum, we have to maximize variance

1. Problem PCA Solves

You have high-dimensional data with many features. Problems include redundant features, noise, slow models, and hard visualization. Goal is to reduce dimensions and keep maximum information. Core idea is to find new axes where data variance is maximum.

---

2. Key Intuition

PCA performs rotation of axes, not deletion of data. Old axes represent original features, new axes represent directions of maximum variance. You project data onto these new axes.

PCA finds new axes (called principal components) that point in the directions of maximum variance in your data. You can think of it as turning your coordinate system so that the first axis captures the most variation, the second captures the next most, and so on.

---

3. Variance (Why important)

Variance means how spread out data is. High variance means more information, low variance means less useful information. PCA keeps directions with highest variance and drops directions with low variance.

---

4. Covariance (Relationship between features)

Variance works on one feature, covariance works on two features. Positive means both increase together, negative means one increases and other decreases, zero means no relation. PCA uses this to understand how features move together.

---

5. Covariance Matrix

This matrix shows relationships between all features. Example in 2D:

```
[ var(x)     cov(x,y) ]
[ cov(y,x)   var(y)   ]
```

Diagonal values are variances, off-diagonal values are covariance, matrix is symmetric.

---

6. Correlation vs Covariance

Covariance is scale dependent and hard to compare. Correlation is scaled covariance and lies between -1 and 1. Both give same direction insight. PCA usually uses covariance matrix.

---

7. Linear Transformation (Core concept)

Transformation means changing data space. Matrix performs rotation, stretching, and compression. When you multiply a vector by a matrix, you get a transformed vector. Matrix acts as transformation rule.

---

8. Eigen Vectors and Eigen Values

Eigen vectors are special directions where after transformation direction stays same and only magnitude changes. Eigen values show how much stretching happens. Key equation:

$$A v = \lambda v$$

A is matrix, v is eigen vector, λ is eigen value.

---

9. Why Eigen Things Matter in PCA

Eigen vectors define directions of new axes. Eigen values define importance of each axis. Higher eigen value means more variance and more importance.

---

10. PCA Step-by-Step

Step 1: Mean centering. Subtract mean from each feature to shift data to origin and improve results.

Step 2: Compute covariance matrix using:

```
np.cov(X.T)
```

Step 3: Perform eigen decomposition to find eigen values and eigen vectors.

Step 4: Sort eigen values in descending order and pick top k.

Step 5: Select top eigen vectors. These become principal components.

Step 6: Transform data using projection:

```
X_new = X · V
```

where V contains selected eigen vectors.

---

11. Dimensionality Reduction

You reduce dimensions like 3D to 2D to 1D. Number of components depends on how much variance you want to retain.

---

12. Explained Variance

This shows how much information each component keeps. Formula:

```
explained_variance = eigen_value / total_eigen_values
```

Used to decide number of components.

---

13. Code Example (Minimal)

```
import numpy as np

X = np.array([[2, 3],
              [3, 4],
              [4, 5]])

X_meaned = X - np.mean(X, axis=0)

cov_matrix = np.cov(X_meaned.T)

eigen_values, eigen_vectors = np.linalg.eig(cov_matrix)

idx = np.argsort(eigen_values)[::-1]
eigen_vectors = eigen_vectors[:, idx]

V = eigen_vectors[:, :1]

X_reduced = np.dot(X_meaned, V)

print(X_reduced)
```

---

14. Important Points for Exams

PCA is unsupervised, PCA is linear, PCA is sensitive to scaling so use standardization, PCA works best when features are correlated.

---

15. When to Use PCA

Use PCA when you have too many features, features are correlated, visualization is needed, or speed optimization is needed. Avoid PCA when interpretability is critical or features are already few.

==========

DAY 49: PRINCIPLE COMPONENT ANALYSIS (PCA)

PCA of MNIST Dataset along with code:

```
import pandas as pd
import matplotlib.pyplot as plt

 ====================== 1. LOAD DATA ======================
Read MNIST CSV
df = pd.read_csv('/kaggle/input/datasets/animatronbot/mnist-digit-recognizer/train.csv')


# Dataset:
# column 0 = label (digit)
# columns 1–784 = pixel values (28x28 image flattened)


# ====================== 2. SPLIT INTO X AND Y ======================
# X = features (pixels)
X = df.iloc[:, 1:]

# Y = target (digit label)
Y = df.iloc[:, 0]


# ====================== 3. TRAIN-TEST SPLIT ======================
from sklearn.model_selection import train_test_split

x_train, x_test, y_train, y_test = train_test_split(
    X, Y,
    test_size=0.2,       # 20% test data
    random_state=42      # fixed shuffle → same split every run
)

# random_state:
# fixes randomness → reproducible results


# ====================== 4. KNN BEFORE SCALING ======================
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

knn = KNeighborsClassifier()

# KNN stores training data (no real training phase)
knn.fit(x_train, y_train)

# Predict using nearest neighbors
y_pred = knn.predict(x_test)
```

```
# Check accuracy
print("Accuracy before scaling:", accuracy_score(y_test, y_pred))


# ====================== 5. FEATURE SCALING ======================
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

# fit → learn mean and std from training data
# transform → apply scaling
x_train = scaler.fit_transform(x_train)
x_test = scaler.transform(x_test)

# scaling:
# converts data to mean=0, std=1
# prevents large-value features dominating distance


# ====================== 6. PCA (DIMENSION REDUCTION) ======================
from sklearn.decomposition import PCA

# reduce 784 features → 100 important features
pca = PCA(n_components=100)

# fit → learn principal components
# transform → project data onto new axes
x_train_trf = pca.fit_transform(x_train)
x_test_trf = pca.transform(x_test)

# PCA:
# rotates data → keeps directions with max variance
# reduces noise + computation


# ====================== 7. KNN AFTER PCA ======================
knn = KNeighborsClassifier()

knn.fit(x_train_trf, y_train)

y_pred = knn.predict(x_test_trf)

print("Accuracy after PCA:", accuracy_score(y_test, y_pred))


# ====================== 8. PCA INTERNALS ======================
# eigenvalues → importance of each component
print("Explained variance:", pca.explained_variance_)

# eigenvectors → directions of new axes
print("PCA components shape:", pca.components_.shape)
# shape = (100, 784)
```

```
# ====================== 9. VISUALIZE ONE IMAGE ======================
# take one sample and reshape to 28x28
image = df.iloc[0, 1:].values.reshape(28, 28)

plt.imshow(image, cmap='gray')
plt.title(f"Label: {df.iloc[0,0]}")
plt.show()


# ====================== FINAL FLOW ======================
# image → flatten → scale → PCA → KNN → prediction
```

==========

DAY 50: MACHINE LEARNING ALGORITHMS

1. Simple Linear Regression (Supervised- Regression)

It is easy to understand and foundational algorithm for more advanced algorithms

What is “linear data”

Linear data means:

- When input increases, output changes at a constant rate
- Relationship looks like a straight line or flat surface (hyperlane)
- Linear data exhibits a directly proportional relationship, meaning a constant change in an input variable results in a constant change in the output variable.

Example: If 1 year experience adds 10k salary, then 2 years adds 20k. This is linear behavior

Real data is not perfect.so points scatter around a line.
Goal: Find best fit line that pass closely through all points.

What model does

- Looks at all points
- Tries many lines
- Picks line closest to all points

This is called best fit line.

Types of Linear Regression

- Simple → one input (experience → salary)
- Multiple → many inputs (experience, CGPA → salary)
- Polynomial → curved relationship
- Regularization → prevents overfitting

Equation of a line

$y = m x + b$

- m → slope
- b → intercept

m and b are parameter

and x is input

input These are the values **you give to the model** to get a prediction.

Parameters are the values the **model learns during training**. These values were **not chosen by you**.

Hyperparameters are settings **you choose before training starts**. The model does **not** learn them.

Examples:

- Learning rate
- Number of iterations (epochs)
- Batch size
- Polynomial degree (for polynomial regression)
- Number of neighbors (K in KNN)

- Maximum depth (Decision Tree)

- **Input → Data**
- **Parameter → Learned**

- **Hyperparameter → Chosen**

Understanding slope/gradient (m)

Slope tells how much output changes when input changes. slope tell us weightage of x over y.

Example: m = 10

- x = 1 → y increases by 10
- x = 2 → y increases by 20

Higher slope → stronger effect of x on y
Lower slope → weaker effect on y

Understanding intercept (b)

Intercept means value of y when x = 0

Example
b = 5000

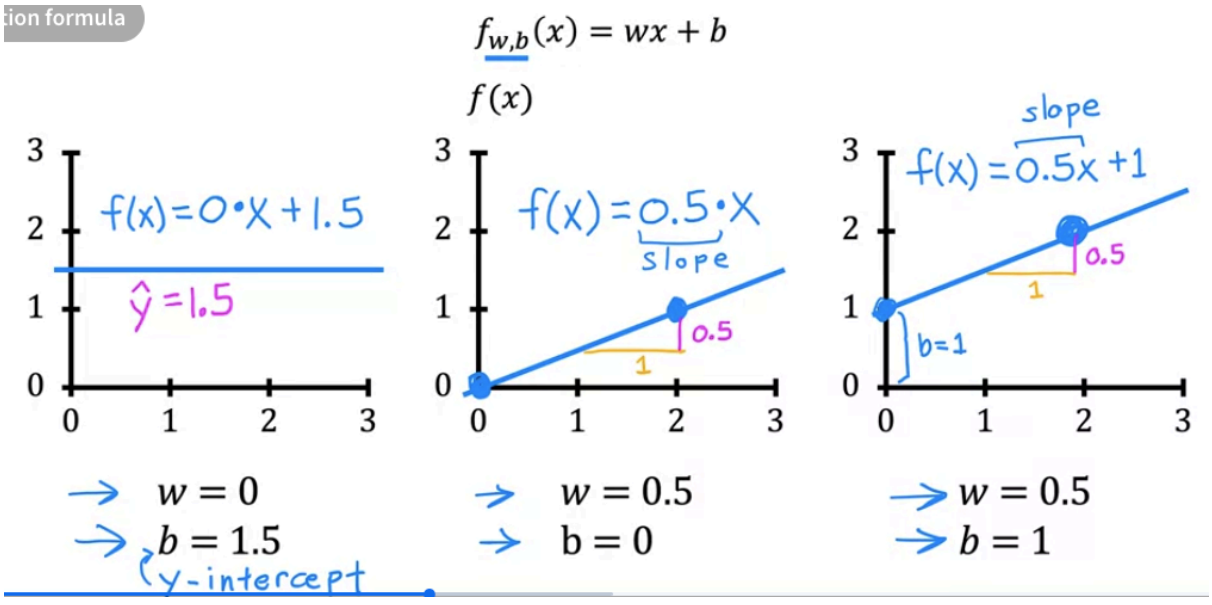
Even if experience = 0
Salary = 5000

This is base value.

Code with explanation

```
import matplotlib.pyplot as plt

# scatter plot of data
# shows real data points
plt.scatter(df['experience'], df['salary'])
plt.xlabel('years of experience')    # x-axis label
plt.ylabel('salary in rs')           # y-axis label

# selecting input (X) and output (y)


x = df.iloc[:, 0:1]   # all rows, first column

# if multiple input columns example: experience + cgpa + age
x = df[['experience', 'cgpa', 'age']]

# target column (output)
y = df.iloc[:, -1]    # last column
```

```
from sklearn.model_selection import train_test_split

# split data into training and testing
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=2
)
from sklearn.linear_model import LinearRegression

# create model
lr = LinearRegression()

# train model on training data
lr.fit(x_train, y_train)

# predict for ONE data point
# reshape is needed because model expects 2D input
lr.predict(x_test.iloc[0].values.reshape(1, 1))

# predict for ALL test data
y_pred = lr.predict(x_test)


# visualize best fit line

plt.scatter(df['experience'], df['salary'])

# plot prediction line using training data
plt.plot(x_train, lr.predict(x_train), color='red')
plt.xlabel('years of experience')
plt.ylabel('salary in rs')

# get slope (m)
m = lr.coef_

# get intercept (b)
b = lr.intercept_

# equation y = m*x + b
```



Question answered about linear regression misconception

1. What is Linear Regression?

Linear regression is a supervised machine learning algorithm used to predict a **continuous numerical value**.

Example:

```
Input (X)           Output (Y)

House Size  -----> House Price
```

The goal is to learn the relationship between input features and the output.

---

2. Does Linear Regression Draw One Line or Many?

❌ Misconception

One line for every training example.

✅ Correct

Linear regression learns **only one best-fit line** (or one hyperplane if there are multiple features).

Example Training Data

Size	Price
1000	200
1200	240
1500	300
1800	360
2000	400

Graph

```
Price
 ^
 |
400|                        •
360|                    •
300|               •
240|          •
200|     •
 |
 +---------------------------------> Size
```

The algorithm finds **one line** that best fits all points.

---

3. What Does the Model Learn?

The model learns an equation.

For one feature: $\hat{y} = mx+b$

where

- x = input feature
- $\hat{y}$ = predicted value
- m = slope (weight)
- b = intercept (bias)

Example: $\hat{y} = 0.2x$

This equation is called the **trained model**.

---

4. What Happens During Prediction?

- Suppose the learned equation is: $\hat{y} = 0.2x$
- New house: `Size =1700`
- Prediction. $\hat{y} = 0.2 \times 1700 = 340$
- The model simply substitutes the input into the equation. No retraining.No new line.

5. Where Does the Predicted Value Lie?

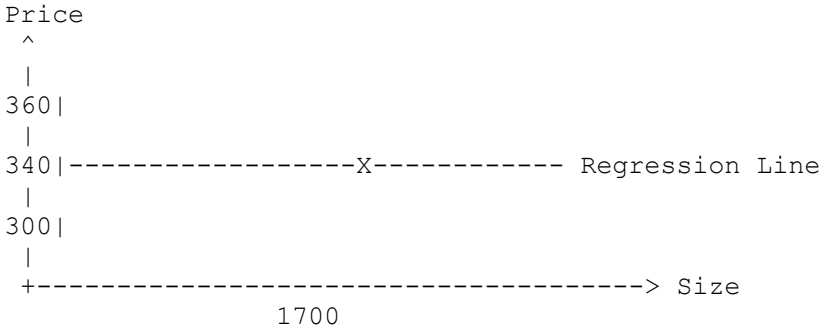
Important Concept

Every predicted value lies **on the regression line**.

Example

- Model: $\hat{y} = 0.2x$
- Input: `1700`
- Prediction: `340`

Graph



The point

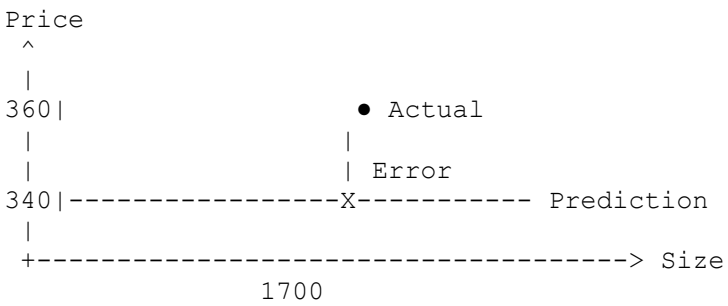
`(1700,340)`

is on the line.

6. Where Does the Actual Value Lie?

Suppose the real price is: `360`

Graph



The actual value is **not required to be on the line**. Only the prediction lies on the line.

---

7. What is Error?

Error is calculated **for one individual data instance**.

Formula: `Error = Actual - Predicted`

Example: Actual: `360`, Prediction: `340`

Error: `20`

Every sample has its own error.

---

8. Why Don't We Add Errors Directly?

Suppose

- `Errors`
- `50`
- `-50`
- `20`
- Adding gives

- `20`
- This looks small even though the model made two huge mistakes.
- Positive and negative errors cancel each other.
- Therefore we square them.

---

9. Squared Error

Instead of

`50`

`-50`

`20`

We calculate

$50^2 = 2500$

$(-50)^2 = 2500$

$20^2 = 400$

Now every error is positive.

---

10. What is the Cost Function?

Most Important Concept

Cost function is **NOT** calculated for one sample.

Cost function is calculated using **all training samples (or a batch)**.

- A **large cost** means the line fits the training data poorly.
- A **small cost** means the line fits the training data well.
- **Gradient descent's only job is to keep changing www and bbb until this cost becomes as small as possible.**

So, the **meaning** of the cost is more important than its exact value. A cost of **100** by itself doesn't tell you whether the model is good or bad—you interpret it by comparing it to previous iterations or to other models trained on the same problem.

Example

Training Data

Actual	Prediction
200	190
300	320
400	390

Errors

```
10
-20
10
```

Square them

```
100
400
100
```

Add

```
600
```

Mean Squared Error

```
600/3
=200
```

Linear Regression Cost: J= 1/2m (sum(y- ŷ)^2)

Here: `m=3`

Cost: `600/(2×3) =100`

This single value represents how well the model is performing overall.

---

11. Difference Between Error and Cost

Error	Cost Function
For one sample	For the whole dataset (or a batch)
Actual − Prediction	Average of squared errors
Many errors exist	Only one cost value per dataset/batch

Example

Training data has

```
100 houses
```

There are

```
100 individual errors
```

But

```
1 cost value
```

---

12. How Does Linear Regression Learn?

Step 1

Start with a random line.

Example $\hat{y} = 0.1x$

Step 2

Predict every training sample.

Actual	Prediction
200	180
300	260
400	380

Step 3

Calculate every error.

```
20

40

20
```

Step 4

Square every error.

```
400

1600

400
```

Step 5

Calculate one cost value.

Step 6

Gradient Descent changes

- slope
- intercept

to reduce the cost.

Step 7

Repeat thousands of times until the cost becomes very small.

Final equation

$\hat{y}=0.2x$

Training is finished.

---

13. What Happens During Testing?

The line is already learned.

Nothing changes.

Example

Test house

`Size=1700`

Prediction

`340`

Suppose actual price

`360`

Error

```
20
```

Repeat for every test sample.

Now calculate

- MSE
- RMSE
- MAE
- R^2

These metrics measure how good the trained model is on unseen data.

The model is not updated using the test set.

---

14. Training vs Testing

Training

```
Training Data
      |
      ▼
Draw the Current regression Line
      |
      ▼
Make Predictions
      |
      ▼
Errors
      |
      ▼
Cost Function
      |
      ▼
Update Line through gradient descent
      |
      ▼
Repeat until we get final w and b
```

Goal

```
Learn the best line.
```

Testing

```
Test Data
      |
      ▼
Final Learned Line
      |
      ▼
Predictions
      |
      ▼
```

```
Compare with Actual
      |
      ▼
Evaluation Metrics
```

Goal

```
Measure performance.

DO NOT update the model.
```

---

15. Common Misconceptions (Cleared)

❌ One line is made for every house.

✅ No.

One best-fit line is learned from all training data.

❌ Test data is used to learn the line.

✅ No.

Only training data is used to learn the line.

❌ Actual test points lie on the regression line.

✅ No.

Only predicted points lie on the line.

Actual points may be above or below it.

❌ Cost function is calculated for one sample.

✅ No.

Error is for one sample.

Cost is calculated using all samples (or a batch).

❌ Test data has no error because predictions come from the line.

✅ Incorrect.

Predictions always come from the line, but the **actual** test values usually differ.

Example

```
Prediction =340

Actual =360

Error =20
```

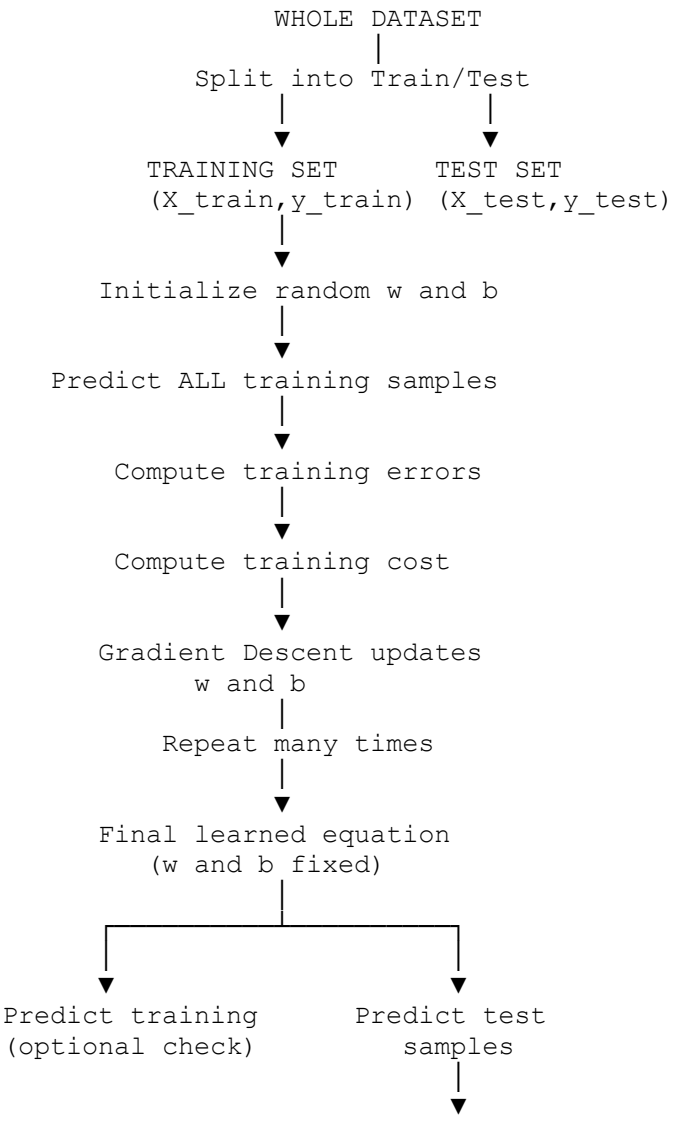
❌ We calculate cost only on the training set.

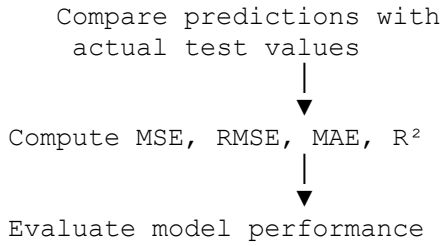
✅ During **training**, the cost function is used to **optimize** the model.

During **testing**, you can compute the same type of quantity (e.g., MSE), but it is **only for evaluation**, not for updating the model.

---

16. The Complete Linear Regression Pipeline





The single sentence that will keep your concepts clear is:
Training data is used to learn and optimize the model (by minimizing the cost), while test data is used only to evaluate how well the already-trained model performs. The test data never changes the learned weights (`w`) or bias (`b`).

Final Summary (The 6 Sentences to Remember)

1. **Linear regression learns one best-fit line from the training data.**
2. **Every prediction is obtained by substituting the input into the learned equation.**
3. **Predicted values always lie on the regression line.**
4. **Actual values may be above or below the line, and their difference from the prediction is the error (residual).**
5. **Each sample has its own error, but the cost function combines the errors of all training samples into one value that the model minimizes during training.**
6. **After training, the learned line is fixed. The test set is only used to evaluate the model—it does not change the line.**
7. **We optimized cost in the training data case or test data case? Only on the training data.**

==========

DAY 51: MACHINE LEARNING ALGORITHMS

1. Simple Linear Regression (Mathematical Intuition)

1. What is Simple Linear Regression

You try to draw a **straight line** that best fits the data.

Equation: Y = mX + b

- **m** = slope → how steep the line is
- **b** = intercept → where line cuts Y-axis

---

2. Goal of Linear Regression

- we want Predicted value close to actual value

- we want Line that fits **most points properly**

---

3. Closed Form Solution (OLS)

Closed form means: You directly compute **m and b using formula**, No iteration, No guessing

Example:

- You plug values into formula
- You get exact answer

This is called: **OLS (Ordinary Least Squares)**

Used when: Data is small and Dimensions are low (like 1 feature)

---

4. Gradient Descent (Non-Closed Form)

- No direct formula
- Start with random values , you guess initial values Example: m = 0 b = 0
- Improve step by step, we repeat until error become small by adjusting m and b vlaues

At each step:

- You calculate error using MSE
- You check direction:
  - Increase m?
  - Decrease m?
- You move in direction where error decreases

Used when:

- Data is large
- Many features (high dimension)

---

5. Scikit-learn Insight

- `LinearRegression` → uses **OLS internally**
- `SGDRegressor` → uses **Gradient Descent**

---

6. Mathematical Intuition of m (Slope)

Formula:

$$m = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sum(x_i - \bar{x})^2}$$

Meaning:

- Numerator → how x and y move together
- Denominator → spread of x (How much the values of **X are scattered or far from their average)**

If:

- x increases and y increases → m positive
- x increases and y decreases → m negative

Simple intuition about data spread

- If values are close to each other → **low spread**
- If values are far apart → **high spread**

Example: Low spread

X = [2, 3, 4] : Values are close and Small variation

High spread

X = [1, 10, 100] : Values are far apart , Large variation

---

7. Finding b (Intercept)

Once m is known:

$$b = \bar{y} - m\bar{x}$$

Simple:

- Take mean of y and x
- Subtract effect of slope

---

8. Loss Function / Cost Function

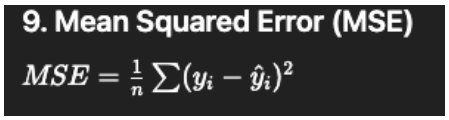
You need a way to measure error.

Error = y{actual} - y{predicted}

Cost function = overall error of model

---

9. Mean Squared Error (MSE)



---

10. Why we use Square (not absolute)

Important concept.

We took square so that opposite distance didn't cancel out each other and for that we don't use absolute because we want to penalize the outliers and we have to differentiate and mod absolute is not differentiable.

If we use square:

- Large errors become very large
- Model focuses on big mistakes
- Smooth function → easy math

If we use absolute:

- All errors treated equally
- Harder to optimize mathematically

So:

- MSE is easier for optimization
- Works well with OLS

---

11. Your Code Explanation (Manual Linear Regression)

```
class MeraLR:

    def __init__(self):
        self.m = None
        self.b = None

    def fit(self, X_train, y_train):

        num = 0
        den = 0

        # Loop through all data points
        for i in range(X_train.shape[0]):
```

```
            # numerator: covariance part
            num = num + ((X_train[i] - X_train.mean()) * (y_train[i] -
y_train.mean()))

            # denominator: variance part
            den = den + ((X_train[i] - X_train.mean()) * (X_train[i] -
X_train.mean()))

        # slope
        self.m = num / den

        # intercept
        self.b = y_train.mean() - (self.m * X_train.mean())

        print(self.m)
        print(self.b)

    def predict(self, X_test):
        return self.m * X_test + self.b
```

Code Intuition

fit()

- You calculate **m manually**
- Using formula (covariance / variance)

Step by step:

- Loop over all points
- Compute numerator and denominator
- Divide to get slope

Then: Compute **b using mean formula**

predict()

- Apply equation: $y = mx + b$
- For new input → get prediction

---

13. Key Takeaways

- Linear Regression = fit best straight line
- OLS = direct formula method
- Gradient Descent = iterative method
- **m = relationship between x and y**
- **b = starting point of line**
- MSE = error measurement
- Squaring error makes model care about large mistakes

==========

DAY 52: MACHINE LEARNING ALGORITHMS

1. Regression Metrics (Linear Regression)

- Input: CGPA
- Output: Salary

Model gives predictions. Now question: **“Is this model good or bad?”**

You need a **number** to judge.

Without metrics:

- You only see a line
- You cannot compare models

With metrics:

- You get a score
- You compare models
- You improve models

---

3. Error function vs Loss function

Error

- Difference for one data point
- Example:
  $y - \hat{y}$

For one point:

- Real salary = 50k
- Predicted = 40k

Error = 10k

This is raw difference.

Loss function

- Calculates Error for one data point after applying formula
- Example:
  $|y - \hat{y}|$ or $(y - \hat{y})^2$

You transform error into something usable.

Example:

- Take absolute → $|10| = 10$
- Square it → 100

Why transform?

Because:

- Raw error cancels out (+ and -)
- You want **magnitude of mistake**

Cost function

- Calculates the Average loss over all data points/ entire dataset , which the model aims to minimize during training.

Mean Absolute Error (MAE)

Formula:

$$MAE = \frac{1}{n}\sum |y - \hat{y}|$$

What actually happens

For each point:

- Take distance between real and predicted
- Ignore sign
- Add all distances
- Divide by total points

Intuition

Think: "On average, how wrong am I?"

If MAE = 5k: Your prediction is off by 5k on average

Advantages

- Same unit as output
- Easy to understand
- **Robust to outliers** mean outliers do not dominate

Example:

Errors: 2, 3, 4, 100

MAE: (2 + 3 + 4 + 100)/4 = 27.25

Outlier (100) affects result
But not insanely

Disadvantages

- Not differentiable at 0

Why not differentiable matters

Graph of |x| has a sharp corner at 0.

Gradient descent needs smooth curves.
Here slope is undefined at 0.

So optimization becomes harder.

Mean Squared Error (MSE)

$$MSE = \frac{1}{n}\sum(y - \hat{y})^2$$

Meaning

- Squares the error and average it
- Large errors become very large

Intuition: "Punish large mistakes more"

Example:

Errors: 2, 3, 4, 100

MSE: 4 + 9 + 16 + 10000 = huge impact from 100

So: One outlier dominates everything

Advantages

- Differentiable
- Easy for optimization
- Used in most algorithms

Why square helps

- Removes negative sign
- Makes function smooth
- Easy to optimize using gradient descent

Disadvantages

- Unit becomes squared so had to interpret back
- **Penalize outliers: not robust to outliers** : as big errors become extremely big and outliers dominate result

Rule

- More outliers → use MAE
- Clean data → use MSE

Root Mean Squared Error (RMSE)

Formula: RMSE= √MSE

Why we take square root

Problem with MSE:

- Unit becomes squared
- Salary becomes (rupees)² → meaningless

RMSE fixes it: Brings back to original unit

Intuition: Same as MSE but easier to interpret.

If RMSE = 5k: Your error is around 5k

Advantages

- Same unit as output
- Penalizes large errors
- Common in deep learning

Disadvantages

- Still sensitive to outliers , Not robust Because squaring still exists inside so Outliers still dominate

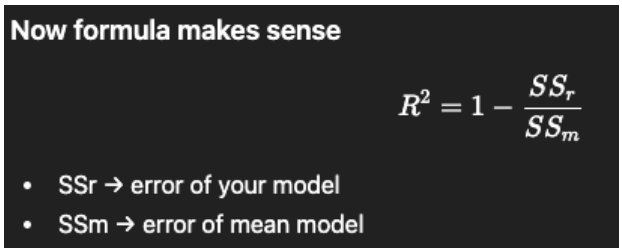
---

R^2 Score (Coefficient of Determination- goodness of fit)

Baseline idea

- Imagine: You ignore model.
- You predict: “Everyone salary = average salary”
- This is called **mean model**
- Now compare: Your regression line vs mean line

What R^2 measures: “How much better is my regression model than just predicting the mean?”



The more we go toward perfectio,n our r2 score move toward 1 because numerator would be zero in that our regression line would be perfect covering all point
The more we towards mean the more our r2 score move towards 0

Cases

Perfect model when $R^2 = 1$

- SSr = 0
- **Means**: Perfect prediction: in this case our regression model does not even make any error, so our numerator become zero which in turn make whole term numerator and this 1 – 0 give us 1 r2 score

Model is same as mean (useless) when $R^2 = 0$

- SSr = SSm
- **Means**: Model useless because regression model is equal to the mean model so there is no need of input features for us.

Worse than mean when $R^2 < 0$ meaning negative

- SSr > SSm
- **Means**: Model is garbage. If r2 score can come negative? Yes, when the ratio of numerator and denominator is greater than 1. In that case it would be SSr > SSm meaning our regression line is doing more error than mean line. Can happen when data is non linear but u apply linear model on it

---

Interpretation

- $R^2 = 1 \rightarrow$ perfect model
- $R^2 = 0 \rightarrow$ same as mean useless
- $R^2 < 0 \rightarrow$ worse than mean when R^2 is negative

Real interpretation

$R^2 = 0.80$

Means: 80% variation explained and 20% unknown

Let say my r2 score came out to 0.80 and I have cgpa and salary, so its interpretation would be cgpa is able to explain 80% of variance in salary, what about remaining 20% we don't know, it could be anything which we couldn't cover mathematically, it tell us how good we doing in regression

Important: It does NOT mean accuracy. It means **explanation power**

Advantages

- Easy to interpret, Independent of unit, Good for model comparison

Disadvantages
Problem is when we add irrelevant input column like temperature to above example, the more input column we introduced the more variance we introducing, so instead of r2 decreasing it sometime increased due to such irrelevant input column, so for that we will study adjusted r2 score.

Problem with R^2

- When you add new feature:
- Example: Add “temperature” to salary model
- Even if useless: R^2 may increase
- Why? Model gets more flexibility, Can fit noise

Adjusted R^2 Score

Formula:

$$Adjusted\ R^2 = 1 - \frac{(1 - R^2)(n - 1)}{(n - 1 - k)}$$

K is no of column/features and n is no of rows and r2 is our normal r2 score

It is designed in such a way that remove the flaw of r2 score that got introduced by adding the irrelevant column
By adding irrelevant column our k will increase which will lower the denominator which in turn will increase the value of right side whole term.so subtracting it from 1, will give us adjusted r2 final score lesser than the r2 score. Also it work in same opposite way when we add some relevant column in that case our adjusted r2 score would be more than r2.
It became a reliable metrics if we dealing with more input data or we dealing in multiple linear regression.
What changes here: **k = number of features**
When you add feature: k increases inturn denominator decreases and thus penalty increases
Intuition: “You only get rewarded if new feature is useful”

- If feature useless: **Adjusted R^2 goes down**
- If useful: **Adjusted R^2 increases**

Final mental model

- MAE → average mistake
- MSE → punish big mistakes
- RMSE → readable MSE
- R^2 → how much model explains
- Adjusted R^2 → real performance with features

Key Comparison

Metric	Outliers	Unit	Use Case
MAE	Robust	Same	Noisy data
MSE	Not robust	Squared	Optimization
RMSE	Not robust	Same	Practical error
R^2	Not about error	Unitless	Model quality
Adjusted R^2	Better than R^2	Unitless	Multiple features

==========

DAY 53: MACHINE LEARNING ALGORITHMS

Multiple Linear Regression (Geometric Intuition)

Multiple Linear Regression extends simple linear regression.

- In simple linear regression: we do have only one input feature
- Equation: y = mx + b

I. In multiple linear regression: we have more than one input features
II. Equation: y = b0 + b1x1 + b2x2 + ... + bnxn

Understanding the Terms

- **y** → target value (output)
- **x1, x2, ..., xn** → input features
- **b0** → intercept (bias)
- **b1, b2, ..., bn** → coefficients (weights)

Meaning of Coefficients

- **b1, b2, ...**
  - Show importance of each feature
  - Higher value → stronger effect on output
  - Tell how much y changes when that feature changes
- **b0 (intercept)**
  - Value of y when all inputs are zero
  - Acts as starting point of prediction

Geometric Interpretation

- 2D (one feature) → line
- 3D (two features) → plane
- 4D or more → hyperplane

So:

- Linear regression fits a **line**
- Multiple regression fits a **plane or hyperplane**

Why It Matters

- Real-world problems rarely depend on one variable
- Multiple regression captures combined effect of many features

Example:

- House price depends on:
  - size
  - location
  - number of rooms

Dataset Generation (Scikit-learn)

You can generate synthetic data using:

- **Scikit-learn function:** `make_regression`

What it does:

- Creates linear datasets
- Supports multiple features
- Useful for practice and testing models

Example idea:

- Set number of features
- Generate X (inputs) and y (output)

==========

DAY 54: MACHINE LEARNING ALGORITHMS

Multiple Linear Regression (Mathematical formulation)

==========

DAY 55: MACHINE LEARNING ALGORITHMS

Multiple Linear Regression (Code)

```
import numpy as np
class MeraLR:
    def __init__(self):
        self.coef_=None
        self.intercept_=None

    def fit(self,X_train,Y_train):
        X_train= np.insert(X_train,0,1,axis=1)
        print(X_train)
        # calculate the coeffs
        betas=np.linalg.inv(np.dot(X_train.T,X_train)).dot(X_train.T).dot(Y_train)
        self.intercept_=betas[0]
        self.coef_=betas[1:]
        # print(betas)

    def predicted(self,X_test):
        y_pred= np.dot(X_test, self.coef_) + self.intercept_
        #Each row → multiply with slopes coefs → sum all → add intercept each row in end
        return y_pred



lr=MeraLR()

lr.fit(X_train,y_train)

y_predict=lr.predicted(X_test)
print(y_predict)

r2_score(y_test,y_predict)

lr.coef_
lr.intercept_
```

==========

DAY 56: MACHINE LEARNING ALGORITHMS

Assumptions of Linear Regression

1. Linear Relationship between input and output

there should be no linear relationship

2. No multicollinearity

- There should be no co-relation between input columns, they should be independent of one another
    - We can detect multicollinearity through variance_inflation_factor
    - 5 or more show multicollinearity
    - We can find it throught the heatmap corr() funtion

3. Normality of residual

The difference between actual and predicted value is called error or residual
When we do predicted and get error and when u plot it, it should be normal distribution

4. Homoscedasticity

Homo means same scedasticity mean spread mean it must have equal spread, when we plot our residual they should be spread equally

5. No autocorrelation of errors

NO AUTOCORRELATION IN THE ERRORS

==========

DAY 57: MACHINE LEARNING ALGORITHMS

Gradient Descent from scratch

OLS (Ordinary Least Squares)

Method

Direct formula. No iteration.

- $w = (X^T X)^{-1} . X^T y$

Computes weights in one step

- Uses matrix inversion

What is happening

- Multiply $X^T X$
- Take inverse of that matrix
- Multiply with $X^T y$
- Get exact best weights

Pros

- Exact solution
- No learning rate
- No epochs
- Same result every time

Cons (important)

Matrix inversion cost:

O(n^3)

- Very slow for large features
- High memory usage
- Fails if matrix is not invertible, If a matrix has no inverse, we call it **non-invertible** (or **singular**). **Why does this happen?**

- The most common reason is **duplicate or perfectly dependent features**.
- Example:

Size	Area in sq ft
1000	1000
1200	1200
1500	1500

- Both columns contain exactly the same information.
- The second feature adds nothing new.Because of this redundancy,
- X^TX cannot be inverted. It mean take x transpose multiply it with x and take determinant and check whether invertible or not
- So the Normal Equation cannot be computed.

Best for

- Small datasets
- Few features

Gradient Descent

Method

Iterative optimization

w = w - \alpha \nabla J

- Start with random weights
- Update step by step

What is happening

Loop:

1. Predict (`y_hat`)
2. Compute error
3. Compute gradient
4. Update weights

Repeat until error reduces

Pros

- Works with huge datasets
- No matrix inversion
- Scales well
- Works with regularization

Cons

- Needs learning rate
- Needs epochs
- Can converge slowly
- Can get stuck if poorly tuned

Best for

- Large datasets
- High-dimensional data
- When data does not fit in memory

Core Difference (1 line)

```
OLS → Solve directly using math
GD  → Learn step by step using error
```

Gradient descent is an optimization technique which take differentiable function and algo give us it minima
Give us minima

General algo used in linear reg, logisitics reg, backbone of deep learning

Types of gd
Batch
Sgd
Mini batch gd

Bnew = b old – slope
We getting drastic change because of this subtracting from slope
So we transform this eq by introducing learning rate

Bnew = Bold – n*slope where n is usually 0.01

Higher learning rate we jumping to opposite side and skipping solution and in lower learning rate we going to solution very slow

So when to stop , how would we know ?

a. difference

B new – b old = 0.001
Agar ye zero howa or close to zero we get then making no improvement and this we converge to solution

b. iterations epoch set to 100 or 1000 or 30 , we stop at correct epoch, epoch mean iteration

Each algoritm has different loss function so will differentiate accordingly , other than we know b , learning rate they would be same

Direction is now a combination of two diff variable m and b that is why it is called gradient and not differentiation

Slope is loss function differentiation either wrt to m or b
Slope is found by differentiating loss function w.r.t to b

Update rule:
$w\_new = w\_old - \alpha \times (dJ/dw)$

Case 1: $dJ/dw = 5$, $\alpha = 0.1$, $w\_old = 8$
$w\_new = 8 - (0.1)(5) = 8 - 0.5 = 7.5$ → gradient positive → w decreases

Case 2: $dJ/dw = -5$, $\alpha = 0.1$, $w\_old = 2$
$w\_new = 2 - (0.1)(-5) = 2 + 0.5 = 2.5$ → gradient negative → w increases

At minimum: $dJ/dw = 0 \rightarrow w\_new = w\_old$ (no change)
-2 sum epsilon (yi-mxi-b)

The Intuition You Should Remember

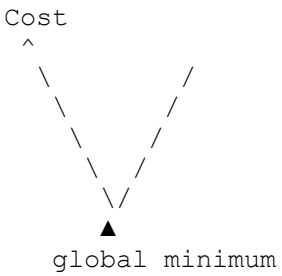
Imagine you're blindfolded on a hill and can only feel the ground beneath your feet.

- If the ground tilts **upward to your right** (positive slope which inturn will make new w smaller), you know the downhill direction is **left**, so you **decrease w**.
- If the ground tilts **upward to your left** (negative slope which add more to new w value), you know the downhill direction is **right**, so you **increase w**.
- If the ground feels **flat** (gradient = 0), you're at the bottom of the valley, where the cost is minimum.

Near a local minimum, derivative get smaller and smaller so the update step also get smaller

Thus we can reach minimum point without decresing learning rate as derivative is decresing and doing the work

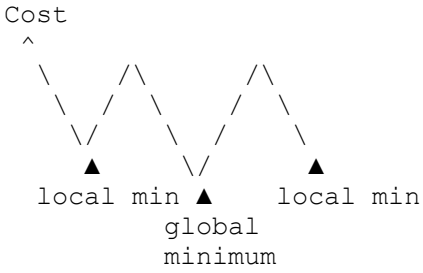
Convex function — has a single bowl shape, one global minimum.



No matter where you start, gradient descent always slides down to the **same lowest point**. There's no risk of getting stuck.

Example: Linear regression's cost function (Mean Squared Error) is convex. This is a big reason MSE is popular — gradient descent is guaranteed to converge to the true minimum.

Non-convex function — has multiple bumps and dips (multiple local minima), not just one.



Here, where you *start* matters. Gradient descent might slide into a **local minimum** — a small dip that looks like the bottom locally, but isn't the true lowest point (the **global minimum**) of the whole function.

Example: Neural network cost functions are almost always non-convex (because of multiple layers and non-linear activation functions). This is why training deep learning models is trickier — you can get stuck in a local minimum, a saddle point, or a plateau.

Key mathematical difference:

A function $J(w)$ is convex if, for any two points w_1, w_2 on the curve, the line segment connecting them never goes below the curve:

$$J(\theta w_1 + (1-\theta) w_2) \le \theta J(w_1) + (1-\theta) J(w_2), \quad \theta \in [0,1]$$

If this holds for **all** points → convex → guaranteed single minimum. If it fails for **some** points → non-convex → possibly multiple minima.

Practical implication for gradient descent:

	Convex	Non-convex
Minima	One (global)	Many (local + global)
Starting point	Doesn't matter	Matters a lot
Guarantee	Always finds the minimum	May get stuck in a local minimum
Example	Linear/logistic regression	Neural networks

That's why techniques like momentum, learning rate scheduling, and random restarts exist — they help gradient descent escape bad local minima in non-convex landscapes.

A better way to say it is:

The derivative tells us how much the loss function changes for a small change in the weight. Based on this information, Gradient Descent updates the weight in the direction that reduces the loss.

Step-by-step

1. We make a prediction:

   $\hat{y} = wx + b$

2. We calculate the loss:

   $L=(\hat{y}-y)2$

3. We compute the derivative:

   $\partial w/\partial L$

   This answers the question:

   "If I change the weight w a tiny bit, how much will the loss change?"

4. We update the weight:

wnew=wold−α∂L/∂w

where α\alphaα is the learning rate.

To check whether its convex or non-convex

What is θ (theta)?

θ is just a **mixing knob** between 0 and 1. It tells you how far you are walking from w_1 to w_2.

$$\theta w_1 + (1-\theta) w_2$$

- θ = 0 → you're standing exactly at w_2
- θ = 1 → you're standing exactly at w_1
- θ = 0.5 → you're standing exactly **halfway** between them

So this formula is just "pick any point on the straight line between w_1 and w_2."

The actual test, in plain words

Pick two points on the curve. Draw a **straight line (chord)** connecting them.

- **Convex** → the chord stays **above or on** the curve everywhere between them.
- **Non-convex** → the chord dips **below** the curve somewhere.

Let me show you visually with actual numbers.## Reading the two panels

Left (convex, $J(w) = w^2$): Pick $w_1 = -2$ and $w_2 = 3$. The dashed chord connects those two points on the curve. At the midpoint ($\theta = 0.5$), the actual curve sits at **0.25**, while the chord sits way up at **6.5**. Curve ≤ chord ✓. This holds for *any* two points you pick — that's what makes it convex.

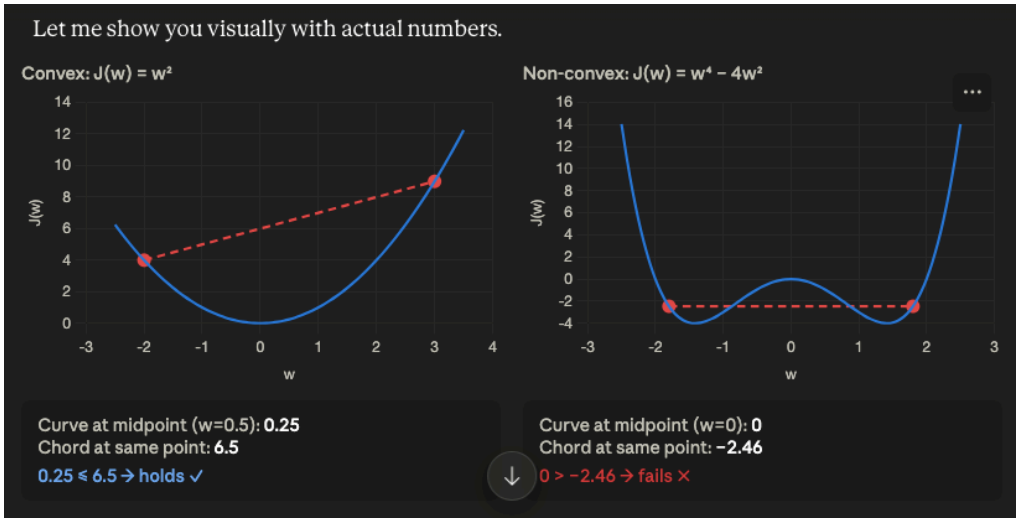
Right (non-convex, $J(w) = w^4 - 4w^2$): This curve has two dips (two local minima) with a bump in the middle. Pick $w_1 = -1.8$ and $w_2 = 1.8$ — both land in the dips, so the chord is a flat line at about **−2.46**. But right in the middle, the actual curve pokes *up* to **0** — sitting **above** the chord. That's the violation: curve > chord means the inequality fails, which is exactly what non-convex means.

Why this matters for gradient descent

The chord test is really just asking: **"does the function ever bulge upward between two points?"**

- If it never bulges (convex) → there's only one valley → wherever gradient descent starts, it will always reach the same bottom.
- If it bulges somewhere (non-convex) → there are multiple valleys separated by a hill → gradient descent can get trapped in whichever valley it starts closest to, even if a deeper valley exists elsewhere.

That's the entire practical reason this definition matters — it's not just abstract math, it's the mathematical guarantee (or lack of one) that gradient descent will find the *true* best answer.



==========

For m it would be
-2 epsilon (yi – mxi -b)xi

Steps

i. initatie random value for m and b
ii. epochs =100 , lr =0.1
    1. for I in epochs:
        a. b=b-n slope
        b. m=m-n slope

b = -100
m = 78.35
lr = 0.1

epochs = 10

```
for i in range(epochs):
    loss_slope = -2 * np.sum(y - m*X.ravel() - b)
    b = b - (lr * loss_slope)
y_pred = m * X + b
plt.plot(X, y_pred)
plt.scatter(X, y)
```

```
class GDRegressor:

    def __init__(self,learning_rate,epochs):
        self.m = 100
        self.b = -120
        self.lr = learning_rate
        self.epochs = epochs

    def fit(self,X,y):
        # calcualte the b using GD
        for i in range(self.epochs):
            loss_slope_b = -2 * np.sum(y - self.m*X.ravel() - self.b)
            loss_slope_m = -2 * np.sum((y - self.m*X.ravel() - self.b)*X.ravel())

            self.b = self.b - (self.lr * loss_slope_b)
            self.m = self.m - (self.lr * loss_slope_m)
        print(self.m,self.b)
```

🎯 Linear Regression:

“I want to find the best line through data”

🧭 Gradient Descent:

“I will slowly adjust the line until it fits best”

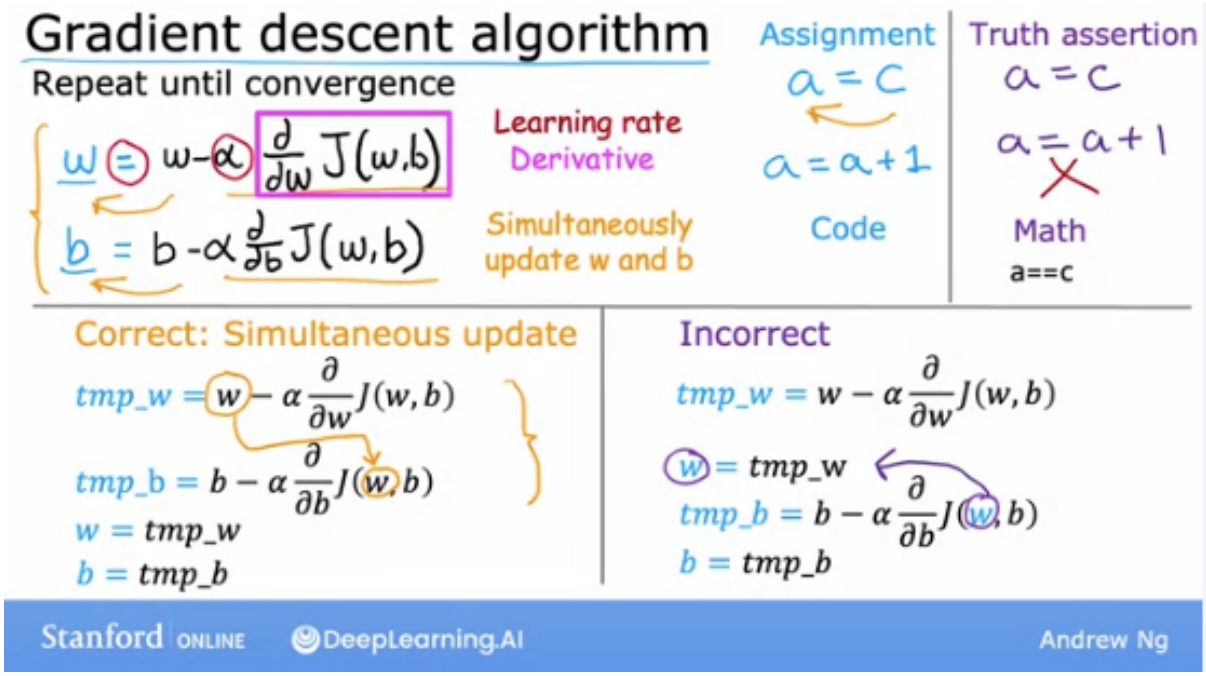
Linear regression defines what we want (best-fit line), and gradient descent is one way to find that best-fit line by iteratively reducing error.

Learning rate impact on gd
Lr small: take much iteration/time to converge
Lr optimum (0.1): take optimum step to find mimima
Lr large (.5): we skip actual target in this and thus bouncing here and there
So in order to find whether our choosen learing rate is optimum or not we should check the geapgh of cost function whether it descresing or not with each iteration,
Better to start with 0.001 then 0.01 then 0.1 and then 1, in this way we try a range of value until we find a reasonable value

Loss function impact on gd

Mean square error
Loacla minima
Global minima
Convex function
Non convez loss function: in deep learning and it has many techniques
Saddle point problem: there comes a plateu a plat surface so it take more time to reach to actual solution and with no enough epochs we never reach to solution

Effect of data
When our data feature are at similar scale then contour plot is so circular and we decsent so fast , so its always better to scale data in linear regression



There are **three main types of Gradient Descent**, and they differ in **how much data they look at before updating the weights**.

1. Batch Gradient Descent (BGD)

Uses the **entire training dataset** to compute the gradient before making a single update.

Example: You have 10,000 house price examples. Batch GD calculates the average gradient across **all 10,000** rows, then updates w once.

- ✅ Very stable, smooth path to the minimum
- ❌ Slow — one update requires a full pass over the data
- ❌ Can't handle datasets too big to fit in memory

2. Stochastic Gradient Descent (SGD)

Uses **just ONE random training example** per update.

Example: Same 10,000 rows. SGD picks row #4521, computes the gradient from just that one row, updates w. Then picks another random row, updates again. This happens 10,000 times per epoch.

- ✅ Very fast updates, works well with huge/streaming data
- ❌ Noisy, zig-zaggy path — doesn't move smoothly downhill
- ✅ The noise can actually help escape shallow local minima (useful in non-convex problems like neural nets)

3. Mini-Batch Gradient Descent

Uses a **small batch** (e.g. 32, 64, 128 examples) per update — a middle ground.

Example: Same 10,000 rows split into batches of 64. Each update uses the average gradient of those 64 examples. ~156 updates per epoch instead of 1 (batch) or 10,000 (SGD).

- ✅ Faster than batch, smoother than SGD
- ✅ This is what's actually used in almost all real deep learning training
- The batch size (32/64/128/256...) is a tunable hyperparameter

Let me show you what their descent paths actually look like:## Quick reference table

Type	Data per update	Path shape	Speed per update	Best for
Batch GD	Entire dataset	Smooth, direct	Slow	Small datasets, convex problems
SGD	1 example	Very noisy zig-zag	Very fast	Huge/streaming data, escaping local minima
Mini-batch GD	Small batch (32–256)	Slightly noisy	Fast	Standard choice — used in almost all deep learning

Simple way to remember it: the more examples you average over before updating, the smoother (but slower) your path downhill. One extreme is "look at everything, then take one careful step" (Batch). The other is "look at one thing, take a step, look at another thing, take a step" (SGD). Mini-batch is the practical compromise everyone actually uses.

==========

DAY 58: MACHINE LEARNING ALGORITHMS

Batch Gradient Descent

Types on basiss of what:

In batch we use total data total rows to update/calculate the slope

It is very slow and have some computational problem

For that we have stockistic

We update based on rows , fast good for large datasets

Mini batch we define a batch

Make me understand the difference between the three types based on some example analogy

Batch is rare and used when we have convex function but there dataset should be big

Stochastic is used widely

Gradient descent applying to multiple linear regreesion

Mathematical formulation:

Y= b0+b1x+b2x

Steps

We start with random values of b0 =0 and slope b1 , b2 = 1

B0=b0 – n slope

B1=b1 – n slope

B2=b2 – n slope

How to calculate the slope of intercept

We have loss function MSE which we gonna expand

So "calculates across all 10,000 rows, then updates once" means:

1. Loop through all 10,000 houses
2. For each one, compute how far off the prediction was

3. Average all 10,000 of those errors into **one number**
4. Use that one number to move ww w **one single time**
5. Repeat — go through all 10,000 rows again for the *next* update

Let's trace it with numbers:

Pass 1:

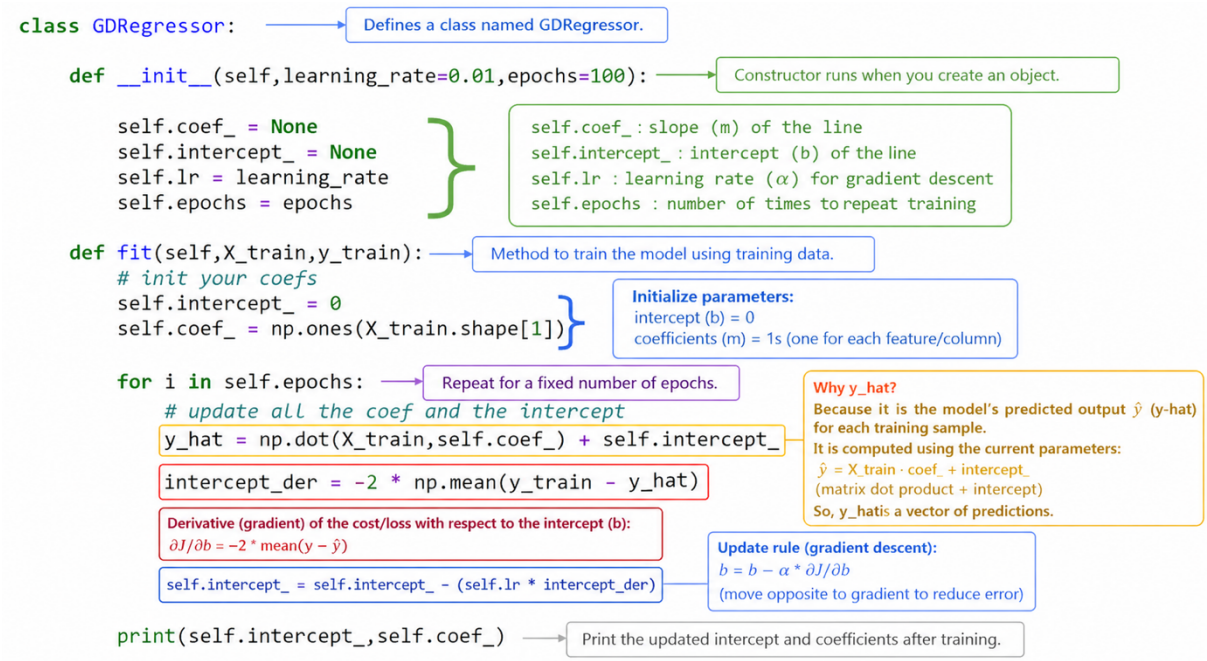
- w=3w = 3 w=3 (starting value)
- House 1 (size = 1000): prediction =w×1000=3000= w \times 1000 = 3000 =w×1000=3000
- Actual price = 2000 → error is large
- ... same for all 10,000 houses → average gradient computed → say it comes out to +8+8 +8
- Update: wnew=3−0.01(8)=2.92w_{new} = 3 - 0.01(8) = 2.92 wnew=3−0.01(8)=2.92

Pass 2:

- ww w is now 2.922.92 2.92 (the *model's* value — not stored anywhere inside the rows)
- House 1 (size = 1000) is **the exact same row as before** — size 1000, price 2000
- But now prediction =2.92×1000=2920= 2.92 \times 1000 = 2920 =2.92×1000=2920 — a *better* prediction than last time, purely because ww w changed
- Repeat for all 10,000 houses → new average gradient → say +5+5 +5
- Update: wnew=2.92−0.01(5)=2.87w_{new} = 2.92 - 0.01(5) = 2.87 wnew =2.92−0.01(5)=2.87

The clean way to say it

The **data stays frozen**. The **model's parameter ww w evolves**. Each pass, you feed the *same* 10,000 rows through the model, but the model itself is a little smarter (has a better ww w) than it was last pass — so the predictions, errors, and resulting gradient are different each time, even though the input data never moved.



Batch GD — Compact Example

Data

```
X = [[1,2],
     [3,4],
     [5,6]]

y = [5,11,17]

w = [1,1], b = 0
```

Step 1: Prediction (use ALL samples)

y_{hat} = Xw + b

```
y_hat = [3, 7, 11]
```

Step 2: Error (ALL samples)

```
error = y - y_hat
      = [2, 4, 6]
```

Step 3: Gradient (core step)

\nabla J = \frac{-2}{n} X^T (y - y_{hat})

Transpose:

```
X^T = [[1,3,5],
       [2,4,6]]
```

Multiply:

```
X^T · error = [44, 56]
```

Final gradient:

```
gradient = [-29.33, -37.33]
```

Key Insight (MOST IMPORTANT)

Each weight uses **ALL samples together**:

```
w1 → (1×2) + (3×4) + (5×6)
      s1       s2       s3
```

No sample is skipped.

Why slow on large data

If 1M samples:

```
w1 gradient =
(x₁×e₁) + (x₂×e₂) + ... + (x₁₀⁶×e₁₀⁶)
```

- huge multiplications per step
- repeated for every feature and epoch

One-line intuition

Batch GD:

```
See ALL errors → take ONE update
```

==========

DAY 59: MACHINE LEARNING ALGORITHMS

Stochastic Gradient Descent

```
Problem with the batch
Let say we have 1000 rows and 5 columns and so 6 coeffs and epoch is 50
```

The Setup

- **Current Weight (w):** 2.0
- **Learning Rate (α):** 0.1

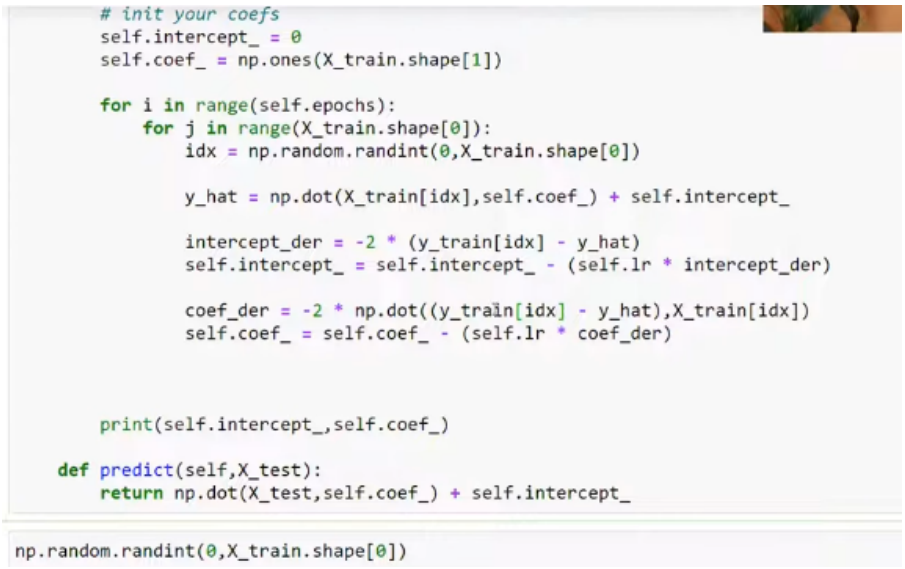
Why this is "Batch" vs "Stochastic":

- **In Batch GD:** You did all that work just to change the weight from **2.0 to 2.033**. You had to look at all 3 rows before the weight moved even a tiny bit.
- **In Stochastic GD:** You would have looked at **Row B**, seen the error was -1, and updated the weight **immediately** to 2.1. Then you would have moved to Row C and updated it again.

The result: Batch is "polite"—it waits for everyone to speak before making a decision. Stochastic is "impulsive"—it reacts to every single person it meets.
Does seeing those numbers help clarify why **Batch GD** is so much slower when you have millions of rows?

Stochastic is fast and require less epoch, give faster convergence , not necessary to load all data to RAM,

We randomly select row and update and that why it called stochastic because of random so it wont be stable



```
        # init your coefs
        self.intercept_ = 0
        self.coef_ = np.ones(X_train.shape[1])

        for i in range(self.epochs):
            for j in range(X_train.shape[0]):
                idx = np.random.randint(0,X_train.shape[0])

                y_hat = np.dot(X_train[idx],self.coef_) + self.intercept_

                intercept_der = -2 * (y_train[idx] - y_hat)
                self.intercept_ = self.intercept_ - (self.lr * intercept_der)

                coef_der = -2 * np.dot((y_train[idx] - y_hat),X_train[idx])
                self.coef_ = self.coef_ - (self.lr * coef_der)


        print(self.intercept_,self.coef_)

    def predict(self,X_test):
        return np.dot(X_test,self.coef_) + self.intercept_
```

```
np.random.randint(0,X_train.shape[0])
```

Used for big data bcuase it converge faster
Used in non convex function : function that got intercepted by ?? , can have local minima and global minima, we can stuck in local minima in case of non convex cost

function, in batch gd but in stochastic we can get out of local minima due to randomness

The problem with stochastic is even near to the solution global minima it still show randomness even though there should be stability , for that come the learning schedule concept in which we vary the learning rate .

==========

DAY 60: MACHINE LEARNING ALGORITHMS

Mini Batch Gradient Descent

Group of rows here , in between batch and stochastic gd

We create batches here like 100 rows and one batch is of 10 rows so total 10 batches we got

Now in each epoch we do updates equal to the number of the batch

```
import random

class MBGDRegressor:

    def __init__(self,batch_size,learning_rate=0.01,epochs=100):

        self.coef_ = None
        self.intercept_ = None
        self.lr = learning_rate
        self.epochs = epochs
        self.batch_size = batch_size

    def fit(self,X_train,y_train):
        # init your coefs
        self.intercept_ = 0
        self.coef_ = np.ones(X_train.shape[1])

        for i in range(self.epochs):

            for j in range(int(X_train.shape[0]/self.batch_size)):

                idx =
random.sample(range(X_train.shape[0]),self.batch_size)

                y_hat = np.dot(X_train[idx],self.coef_) + self.intercept_
                #print("Shape of y_hat",y_hat.shape)
                intercept_der = -2 * np.mean(y_train[idx] - y_hat)
                self.intercept_ = self.intercept_ - (self.lr *
intercept_der)

                coef_der = -2 * np.dot((y_train[idx] - y_hat),X_train[idx])
                self.coef_ = self.coef_ - (self.lr * coef_der)
```

```
        print(self.intercept_,self.coef_)

    def predict(self,X_test):
        return np.dot(X_test,self.coef_) + self.intercept_


mbr =
MBGDRegressor(batch_size=int(X_train.shape[0]/50),learning_rate=0.01,epochs
=100)

mbr.fit(X_train,y_train)
```

==========

DAY 61: MACHINE LEARNING ALGORITHMS

Polynomial Linear Regression

1. What problem it solves

- Linear regression fits a straight line. Some data is curved. A straight line fails.
- Polynomial regression fixes this by adding powers of features.

Example: You try to predict price from size. Relationship curves upward. A straight line misses the pattern. A polynomial curve fits it.

2. Why it is still called linear regression

The trick: You make new features like x^2, x^3, etc but still there is linear relationship between coefficients (slope) with Y

- But the model stays linear in coefficients.
- Form stays: $y = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 \ldots$
- No coefficient multiplies another coefficient. So optimization stays simple.

3. Underfitting vs Overfitting

- **Underfitting**: caused due to low degree, model too simple, it misses patterns, have high bias
- **Overfitting**: caused by high degree, model too complex, Touches almost every point, Learns noise, High variance
- Goal: Find balance

4. Now your case

- 2 input columns, degree = 2
- Inputs: x_1, x_2
- We create polynomial features up to degree 2.

5. Feature transformation (important part)

Original features

x_1, x_2

After polynomial expansion (degree 2):

1
x_1
x_2
x_1^2
x_2^2
x_1x_2

Total features = 6

6. Final model equation

$y=\beta0 + \beta1\ x1 + \beta2\ x2 + \beta3\ x12 + \beta4x22 + \beta5x1x2$

What each term does:

- x_1^2, x_2^2 → capture curvature
- x_1x_2 → captures interaction between features

Example: If study hours and sleep both increase, performance might rise differently than individually. That is interaction.

7. How many features are created

General formula:

- Number of features = (n + d)! / (n! * d!)
- Where: n = number of features , d = degree
- Here: n = 2 and d = 2
- Result = 6 features

8. Small numeric example

- Input: $x_1 = 2$, $x_2 = 3$
- Transformed: 1, 2, 3, 4 (2^2), 9 (3^2), 6 (2×3)
- Now model uses these 6 values to predict y.

---

9. Code example (Python)

Using scikit-learn

```
import numpy as np
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

# sample data (2 features)
X = np.array([
    [1, 2],
    [2, 3],
    [3, 4],
    [4, 5]
])

y = np.array([5, 9, 15, 23])

# create polynomial features (degree 2)
poly = PolynomialFeatures(degree=2)
X_poly = poly.fit_transform(X)

print("Transformed Features:")
print(X_poly)

# train model
model = LinearRegression()
model.fit(X_poly, y)

# prediction
new = np.array([[5, 6]])
new_poly = poly.transform(new)

prediction = model.predict(new_poly)
print("Prediction:", prediction)
```

---

10. Key intuition to remember

- You are not changing the model
- You are changing the input space
- You make data linearly separable in higher dimensions

Simple view:

Original space → curved pattern
New space → becomes linear

---

11. Common mistakes

- Using high degree blindly
- Not scaling features before polynomial expansion
- Ignoring interaction term importance
- Not validating model on unseen data

---

12. When to use

Use when:

- Data shows curve pattern
- Simple linear model fails
- Feature interaction matters

Avoid when: Too many features and Risk of overfitting is high

==========

DAY 62: MACHINE LEARNING ALGORITHMS

Bis Variance Trade off

Bias: Bias is the inability of a ml model to truly capture the relationship in the training data, such model has high bias

Bias = error due to wrong assumptions in the model

- Model is too simple
- Misses real pattern in training data
- Same mistake on most datasets

I. **Example**: You fit a straight line to curved data, Model cannot follow shape
II. **Result**: High training error , and high test error

Variance is basically difference of fits on different datasets like in training dataset the error is 10 and in test dataset, the error is 100 so error difference is 90 which means the variance is high

Variance = how much model changes with different data

- Model is too sensitive
- Learns noise
- Changes a lot if data changes
- **Your example:** Training error = 10, Test error = 100 => Gap = large → high variance
- **Result**: Low training error while High test error

Bias vs Variance intuition

- High bias → model too rigid

- High variance → model too flexible

Goal: Balance both

`Overfitting: work best on training data and not working well on test data`
Overfitting: Model memorizes training data

- Very low training error
- Poor performance on new data

Cause

- High model complexity
- Too many features or high polynomial degree

Variance mtlb performance on one dataset and different performance on different dataset

---

`Underfitting: does not able to capture relationship in training data`
Underfitting: Model fails to learn pattern

- High training error
- High test error

Cause

- Model too simple
- Not enough features

Techniques to handle this

1. Regularization:
- Penalize large coefficients
- Effect: it Reduces model complexity and controls overfitting
- Model is forced to stay smooth

Common types: Ridge Regression and Lasso Regression

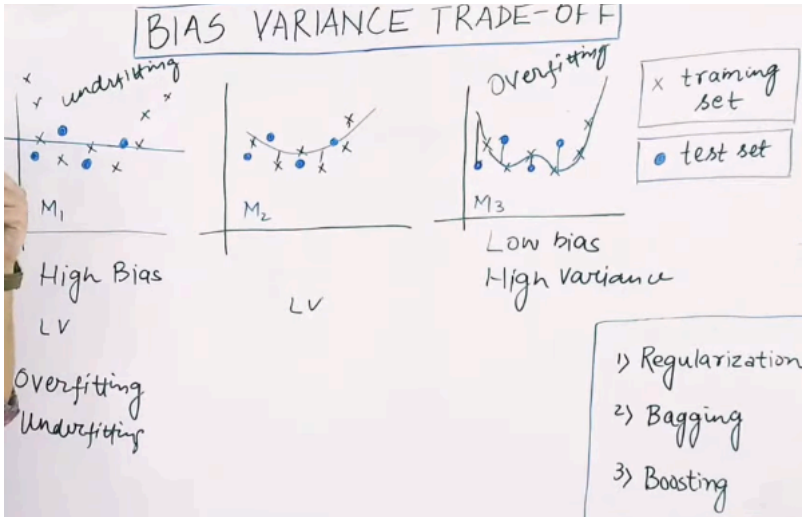
---

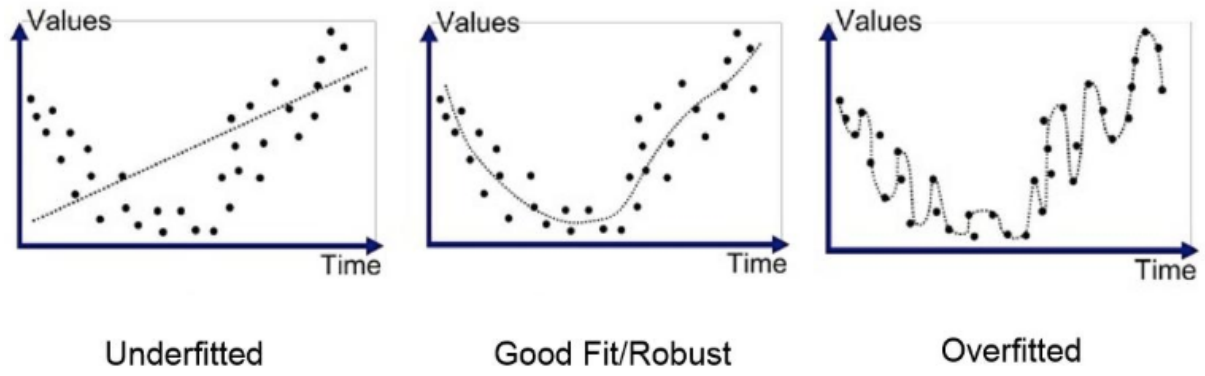
2. Bagging

- **Full form:** Bootstrap Aggregating
- **Idea** is to train many models on different random samples and average their predictions
- Effect: would be Reduces variance
- Example: Random Forest

3. Boosting

- **Idea**: is to train models sequentially where each model focuses on previous mistakes
- **Effect**: Reduces bias and this Improves overall accuracy
- **Examples**: AdaBoost, Gradient Boosting





==========

DAY 62: MACHINE LEARNING ALGORITHMS

Ridge Regression (Regularization)

To reduce overfitting, we add something extra in ml model.

Used in linear regression and logistics regression

There are 3 kinds of regularization technique

Ridge L2
Lasso L1
Elastic net

```
In overfit model m value would be extremely high and in contrast m lower
value would represent underfitting
Lesser slope means there is no role of x in determining the output y
While high slope m means that greater role of x in determining the output y

To reduce overfitting, we had to reduce slope m

Our goal is to minimize the magnitude of loss function so we add
regularization term to that function
Lambda is a hyperparameter which value could be tune lower or high
```

==========

DAY 70: MACHINE LEARNING ALGORITHMS

Logistics Regression (Perceptron Trick)

Why Not Linear Regression for Classification?

Problem 1: Output Range Mismatch

Classification needs a **class label** — usually 0 or 1.

Linear regression produces: $$\hat{y} = wx + b$$

This can output **any real number**: 1.6, -0.2, 5.3, -10... There's no natural way to interpret "class = 1.6" or "class = -0.2." These values simply don't correspond to a class.

Problem 2: "Just Use a Threshold" — Why It Fails

The obvious fix: pick a threshold (e.g., if $\hat{y} \geq 0.5$, predict class 1; else class 0).

This can work for simple, well-separated data. But it breaks down with **outliers** — and here's the mechanism:

- Linear regression's objective is to **minimize squared error**: $\sum (\hat{y}^{(i)} - y^{(i)})^2$

- Squaring means **large errors get punished disproportionately** — an outlier far from the rest of the data contributes a *huge* squared error
- To reduce that huge error, the regression line **tilts toward the outlier**, dragging the decision boundary with it
- Result: a single distant point can **dramatically shift the boundary**, misclassifying points that were previously correctly separated

So the model isn't just "occasionally wrong" — it's **structurally unstable** to outliers because of what it's optimizing for.

Problem 3: Wrong Optimization Objective

Squared error is designed for continuous prediction problems ("how far off was my number?"), not classification ("did I pick the right class?"). It doesn't correspond to a meaningful notion of *classification confidence*.

Why Logistic Regression Fixes This

Fix 1: The Sigmoid Function Bounds the Output

Instead of outputting $\hat{y} = wx + b$ directly, logistic regression passes that value (z) through the **sigmoid function**:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

No matter what z is — even $z = -1000$ or $z = +1000$ — the output is **squeezed into the range (0, 1)**:

$$z \to -\infty \implies \sigma(z) \to 0$$ $$z \to +\infty \implies \sigma(z) \to 1$$ $$z = 0 \implies \sigma(z) = 0.5$$

This output can now be interpreted as a **probability**: "there's an 87% chance this is class 1."

Fix 2: A Loss Function Built for Classification

Logistic regression doesn't use squared error. It uses **log loss (cross-entropy)**:

$$J(w,b) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log(\hat{y}^{(i)}) + (1-y^{(i)})\log(1-\hat{y}^{(i)})\right]$$

This penalizes **confident wrong predictions** heavily (predicting 0.99 when the true label is 0 is punished severely) while being much less sensitive to distant points in the way squared error is — because the penalty is based on probability confidence, not raw distance.

Summary Table

	Linear Regression	Logistic Regression
Output	Any real number	Probability (0 to 1)
Function	$\hat{y} = wx+b$	$\sigma(wx+b)$
Loss	Squared error	Log loss (cross-entropy)
Sensitive to outliers?	Yes — squared error amplifies distant points	Much less — sigmoid saturates, doesn't grow unbounded
Good for	Continuous value prediction	Class probability / classification

We do not usually use MSE for Logistic Regression because, after applying the sigmoid function, the MSE cost becomes non-convex, making gradient descent harder and slower to optimize. Binary Cross-Entropy (Log Loss) is preferred because it produces a convex optimization problem and penalizes incorrect, high-confidence predictions more effectively.

Data would be linear

Two approaches

What is perceptron trick

Perceptron trick: We start with random values of a b and c values.perceptron trick we had to run a loop.generally we run this loop 1000 times or till convergence and in each loop run we choose a random point.if that random point is classified rightly then we don't do anything else we move line. Easily give solution but we won't get best possible solution.

How to identify a -ve and +ve region

Here the line equation would be Ax + By + C = 0

C Is our Y intracept
How can we move line in 2d : by changing a b c values
By changing c our line move in parallel ,
By changing a our line move in axis ratotaion
Bby changing b our line move around some point

Desmos is online calculator to play with a b cand c values

Add 1 in end to the coordinates of some points like (1,3) -> (1,3,1)
and will subtract it from the line values let say 2x + 3y + 5 =0
To move line into negative region we take their addition while in case of moving it into positive region we subtract it from

2x + 3y + 5 =0
1. 3. 1
3x 6y. 6c

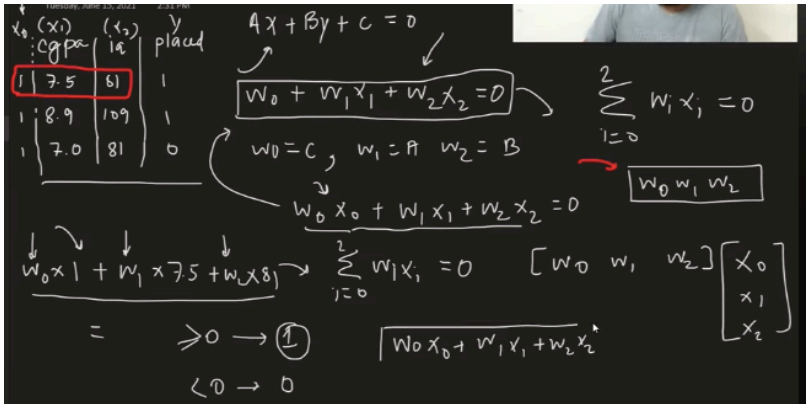
If some -ve points is into some +ve region then we add 1 in last of their coordinaties and subtract it from line coefficient and do same in contrary case too

Learning rate = 0.01
New coef = old coef – n * coordiates n is learning rate

W0 + w1x1 + w2x2 = 0
W0=c, w1=a, w2=b

SUMMATION OF WiXi = 0

ALL GIVE NOTES OF BELOW EXAMPLE IN SS OR BETTER ONE FROM THAT



If no is positive then prediction is 1 otherwisse 0

Algorithm
We have to decide epoch value generally 100 and decide learning rate which is generally 0.01

For I in range (epoch)
Randomly select a student point
If it belong to +ve and after computatiojn >= 0

Then we new w values

==========

DAY 71: MACHINE LEARNING ALGORITHMS

Logistics Regression (Perceptron Trick Code)

==========

DAY 72: MACHINE LEARNING ALGORITHMS

Logistics Regression (Sigmoid Function)

In before perceptron trick algorithm misclassified points would pull the line
But now correctly classified point would push the line so there would be equlilibruim

These push and pull also depend on distance of point from line s
Missclassified points more away from line would pull with more magnitude and near one would pull slowly with low magnitufde , same goes for classified points pushing

Sigmoid function scales every kind of value in range of 0 to 1

The summation of negative log of maximum likelihood we call it cross entrophy

In maximum likelihood we select amodel whose product of probabilities is highest bu tin cross entrophy we have to minimize it , the best model would be the one who cross trophy is minimum

LOSS FUNCTION OF LOGISTICS REGRESSION

Simplified loss function

$$L\left(f_{\vec{w},b}(\vec{x}^{(i)}), y^{(i)}\right) = \begin{cases} -\log\left(f_{\vec{w},b}(\vec{x}^{(i)})\right) & \text{if } y^{(i)} = 1 \\ -\log\left(1 - f_{\vec{w},b}(\vec{x}^{(i)})\right) & \text{if } y^{(i)} = 0 \end{cases}$$

$$L\left(f_{\vec{w},b}(\vec{x}^{(i)}), y^{(i)}\right) = -y^{(i)}\log\left(f_{\vec{w},b}(\vec{x}^{(i)})\right) - (1 - y^{(i)})\log\left(1 - f_{\vec{w},b}(\vec{x}^{(i)})\right)$$

0 $\qquad$ $(1-0)$

if $y^{(i)} = 1$:

$$L\left(f_{\vec{w},b}(\vec{x}^{(i)}), y^{(i)}\right) = -1\ \log\left(f(\vec{x})\right)$$

if $y^{(i)} = 0$:

$$L\left(f_{\vec{w},b}(\vec{x}^{(i)}), y^{(i)}\right) = -(1-0)\log(1-f(\vec{x}))$$

Simplified cost function

loss

$$L\left(f_{\vec{w},b}(\vec{x}^{(i)}), y^{(i)}\right) = -y^{(i)}\log\left(f_{\vec{w},b}(\vec{x}^{(i)})\right) - (1 - y^{(i)})\log\left(1 - f_{\vec{w},b}(\vec{x}^{(i)})\right)$$

cost

$$J(\vec{w}, b) = \frac{1}{m}\sum_{i=1}^{m}\left[L\left(f_{\vec{w},b}(\vec{x}^{(i)}), y^{(i)}\right)\right]$$

$$= -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log\left(f_{\vec{w},b}(\vec{x}^{(i)})\right) + (1 - y^{(i)})\log\left(1 - f_{\vec{w},b}(\vec{x}^{(i)})\right)\right]$$

maximum likelihood

==========

More features and less data also result in the overfitting.
One way to cure overfitting is to add more data
Select features selction
Reduce size of parameter regularization